

CW2_Data Analytics

Exploratory Data Analysis & Machine Learning

Priyanshu Thakur

May 05, 2026

Contents

Setup	3
Installing & Loading Packages	3
Importing the Dataset	3
Exploratory Data Analysis & Data Preparation	3
Initial Inspection	3
Cross-Tabulations	5
Buying vs. Acceptability	5
Maintenance vs Acceptability	5
Doors vs. Acceptability	6
Persons vs. Acceptability	6
lug_boot vs. Acceptability	6
Safety vs. Acceptability	7
Visualising Variable Distributions	7
Acceptability	7
Buying price	8
Safety	10
Persons	11
Doors	11
Feature Relationships	12
Mutual Information Matrix	12
Chi-Squared Tests	15
Buying vs. Acceptability	15
Buying vs. Maintenance	15
lug_boot vs. Acceptability	15
Safety vs. Acceptability	16

Train / Test Split	16
Verifying the Split	16
Machine Learning Algorithms	17
The Universal ML Workflow	17
Decision Tree (C5.0)	17
Training & Prediction	17
Evaluation	18
Naive Bayes	19
Multinomial Logistic Regression	20
Support Vector Machine (SVM)	21
Neural Network	22
Model Comparison	23
Baseline Accuracy Summary	23
10-Fold Cross-Validation	24
Decision Tree with CV	25
Naive Bayes with CV	26
Logistic Regression with CV	26
Neural Network with CV	27
SVM with CV	28
Cross-Validation Summary	29
Filter-Based Feature Selection	30
Step 1 : Recompute MI on the training set only	30
Step 2 : Build feature-selected datasets	31
Step 3 : What to look for	31
Decision Tree Selected Features	31
Naive Bayes Selected Features	32
Logistic Regression Selected Features	33
SVM Selected Features	34
Neural Network Selected Features	35
Bonus — Top 4 features	36
Feature Selection Summary	37

Improving Model Performance — Hyperparameter Tuning	38
Neural Network Tuning	38
Decision Tree Tuning	39
Naive Bayes Tuning	40
Logistic Regression Tuning	41
SVM (Linear) Tuning	42
Tuned Model Summary	43
Conclusion	45
References	46

Setup

Installing & Loading Packages

The analysis uses a range of packages for data handling, visualisation, and modelling. The snippet below installs any that are missing and loads them all silently.

```
packages = c("nnet", "caret", "class", "gmodels", "C50",
             "klaR", "e1071", "kernlab", "caTools", "ggplot2", "infotheo",
             "dplyr", "reshape2", "knitr", "scales")

new.packages = packages[!(packages %in% installed.packages()[, "Package"])]
if (length(new.packages)) install.packages(new.packages)

invisible(lapply(packages, library, character.only = TRUE))
```

Importing the Dataset

The UCI Car Evaluation dataset contains **1000 records** with six categorical features and one target variable (acceptability).

```
source_file = "car.csv"
df = read.csv(source_file)
```

Exploratory Data Analysis & Data Preparation

Initial Inspection

```
head(df, 10)
```

```
##      buying maint doors persons lug_boot safety acceptability
## 1    vhigh   low 5more         4    small   low         unacc
## 2     high vhigh    3         2     med    low         unacc
## 3    vhigh   high    4         4     big    med         unacc
## 4     high vhigh 5more         4     med    low         unacc
## 5    vhigh   high 5more         2     med    high        unacc
## 6     med vhigh    4    more    small    med         unacc
## 7     med   med    4     2     big    med         unacc
## 8     low vhigh    2    more    big    high          acc
## 9     med   low    4     4    small    med          acc
## 10    med   low    4    more    big    med          good
```

```
cat("Dimensions:", nrow(df), "rows", ncol(df), "columns\n")
```

```
## Dimensions: 1000 rows 7 columns
```

```
cat("Missing values:", sum(is.na(df)), "\n\n")
```

```
## Missing values: 0
```

```
df =data.frame(lapply(df, factor))
```

```
summary(df)
```

```
##      buying      maint      doors      persons      lug_boot      safety
## high :270    high :242    2      :243    2      :321    big   :341    high:330
## low  :238    low  :248    3      :246    4      :343    med   :324    low  :337
## med  :257    med  :259    4      :259    more:336    small:335    med  :333
## vhigh:235    vhigh:251    5more:252
## acceptability
## acc  :231
## good : 41
## unacc:693
## vgood: 35
```

```
str(df)
```

```
## 'data.frame':    1000 obs. of  7 variables:
## $ buying      : Factor w/ 4 levels "high","low","med",...: 4 1 4 1 4 3 3 2 3 3 ...
## $ maint       : Factor w/ 4 levels "high","low","med",...: 2 4 1 4 1 4 3 4 2 2 ...
## $ doors       : Factor w/ 4 levels "2","3","4","5more": 4 2 3 4 4 3 3 1 3 3 ...
## $ persons     : Factor w/ 3 levels "2","4","more": 2 1 2 2 1 3 1 3 2 3 ...
## $ lug_boot    : Factor w/ 3 levels "big","med","small": 3 2 1 2 2 3 1 1 3 1 ...
## $ safety      : Factor w/ 3 levels "high","low","med": 2 2 3 2 1 3 3 1 3 3 ...
## $ acceptability: Factor w/ 4 levels "acc","good","unacc",...: 3 3 3 3 3 3 3 1 1 2 ...
```

Key takeaway: There are no missing values. All variables are categorical, which makes this an ideal dataset for classification algorithms that handle factors natively (e.g., C5.0, Naive Bayes).

Cross-Tabulations

`table()` lets us cross-tabulate categorical variables and spot patterns quickly.

Buying vs. Acceptability

```
df$acceptability = factor(df$acceptability, levels = c("unacc", "acc", "good", "vgood"))
df$buying = factor(df$buying, levels = c("vhigh", "high", "med", "low"))

knitr::kable(
  addmargins(table(df$buying, df$acceptability)),
  caption = "Buying vs. Acceptability"
)
```

Table 1: Buying vs. Acceptability

	unacc	acc	good	vgood	Sum
vhigh	195	40	0	0	235
high	198	72	0	0	270
med	162	68	14	13	257
low	138	51	27	22	238
Sum	693	231	41	35	1000

Insight: Low-price cars are more commonly rated as acceptable, suggesting price can be a key driver of acceptability.

Maintenance vs Acceptability

```
df$maint = factor(df$maint, levels = c("vhigh", "high", "med", "low"))

knitr::kable(
  addmargins(table(df$maint, df$acceptability)),
  caption = "Maintenance vs. Acceptability"
)
```

Table 2: Maintenance vs. Acceptability

	unacc	acc	good	vgood	Sum
vhigh	212	39	0	0	251
high	170	68	0	4	242
med	151	76	14	18	259
low	160	48	27	13	248
Sum	693	231	41	35	1000

Insight: low maintenance cars are rated as acceptable, suggesting maintenance can be a key driver of acceptability.

Doors vs. Acceptability

```
df$doors = factor(df$doors, levels = c("2", "3", "4", "5more"))

knitr::kable(
  addmargins(table(df$doors, df$acceptability)),
  caption = "Doors vs. Acceptability"
)
```

Table 3: Doors vs. Acceptability

	unacc	acc	good	vgood	Sum
2	183	46	8	6	243
3	172	57	10	7	246
4	175	63	11	10	259
5more	163	65	12	12	252
Sum	693	231	41	35	1000

Insight: 5more doors cars are more commonly rated as acceptable, suggesting doors can be a key driver of acceptability.

Persons vs. Acceptability

```
df$persons = factor(df$persons, levels = c("2", "4", "more"))

knitr::kable(
  addmargins(table(df$persons, df$acceptability)),
  caption = "persons vs. Acceptability"
)
```

Table 4: persons vs. Acceptability

	unacc	acc	good	vgood	Sum
2	321	0	0	0	321
4	188	116	20	19	343
more	184	115	21	16	336
Sum	693	231	41	35	1000

Insight: more persons cars are more commonly rated as acceptable, suggesting persons can be a key driver of acceptability.

lug_boot vs. Acceptability

```
df$lug_boot = factor(df$lug_boot, levels = c("big", "med", "small"))
```

```
knitr::kable(
  addmargins(table(df$lug_boot, df$acceptability)),
  caption = "lug_boot vs. Acceptability"
)
```

Table 5: lug_boot vs. Acceptability

	unacc	acc	good	vgood	Sum
big	226	82	12	21	341
med	214	81	15	14	324
small	253	68	14	0	335
Sum	693	231	41	35	1000

Insight: Big lug_boot cars are more commonly rated as acceptable, suggesting size can be a key driver of acceptability.

Safety vs. Acceptability

```
df$safety = factor(df$safety, levels = c("vhigh", "high", "med", "low"))

knitr::kable(
  addmargins(table(df$safety, df$acceptability)),
  caption = "Safety vs. Acceptability"
)
```

Table 6: Safety vs. Acceptability

	unacc	acc	good	vgood	Sum
vhigh	0	0	0	0	0
high	145	131	19	35	330
med	211	100	22	0	333
low	337	0	0	0	337
Sum	693	231	41	35	1000

Insight: 35 high safety cars are rated vgood, while low and med safety cars have zero vgood ratings. Safety is strongly tied to top ratings.

Visualising Variable Distributions

All figures in this report were produced using the **ggplot2** package (Wickham, 2016).

Acceptability

```
ggplot(df, aes(x = acceptability)) +
  geom_bar(fill = "steelblue", width = 0.6) +
  geom_text(stat = "count", aes(label = after_stat(count)), vjust = -0.5, size = 3.5) +
  labs(title = "Distribution of Car Acceptability",
       x = "Acceptability", y = "Count") +
  theme_minimal(base_size = 13)
```

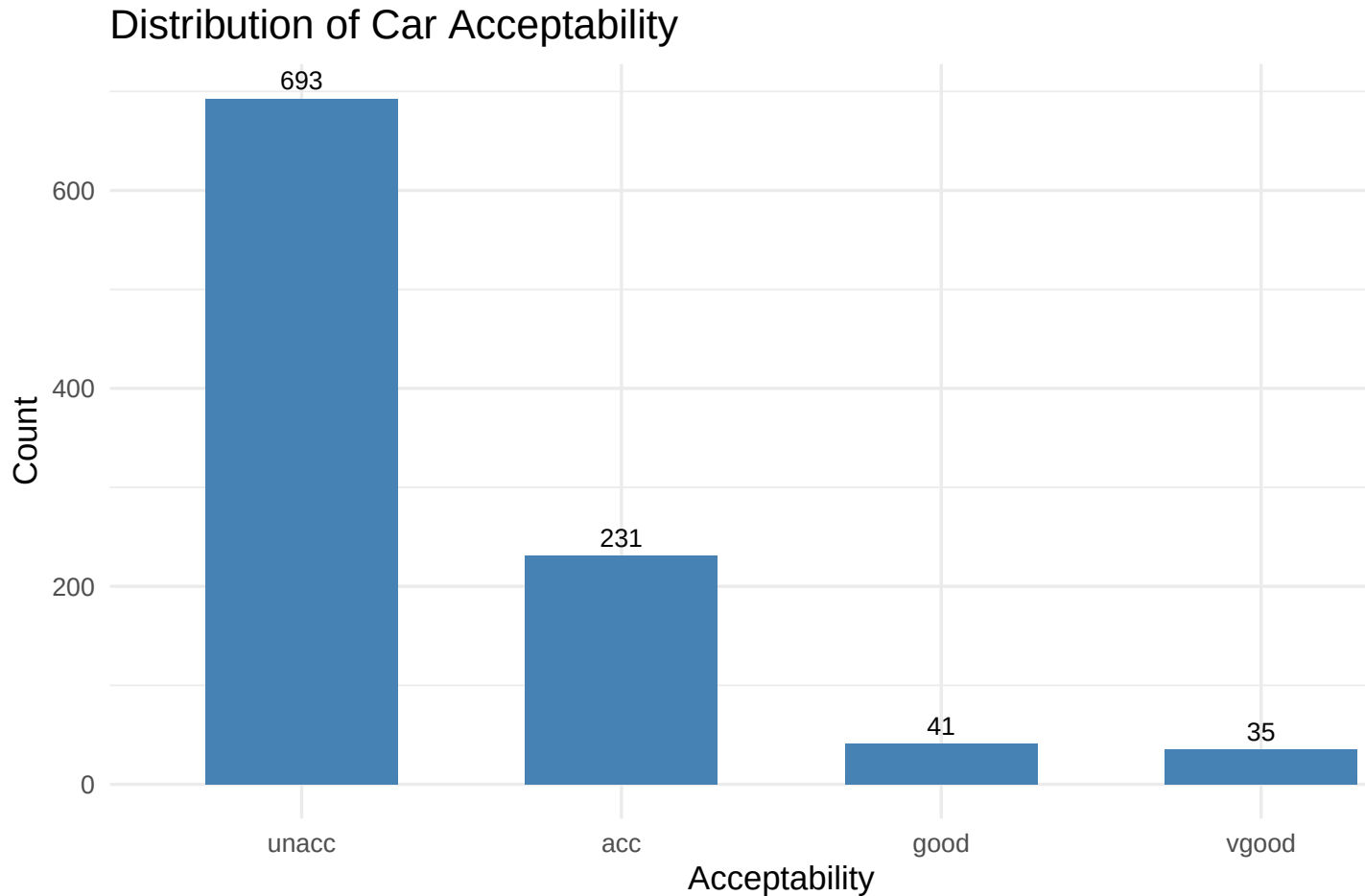


Figure 1: The dataset is heavily imbalanced — 'unacc' dominates.

Description: The target variable is heavily imbalanced. The unacc class dominates while vgood is rarest. This imbalance has direct implications for evaluation overall accuracy alone may be misleading, so per-class metrics such as sensitivity must also be considered (Lantz, 2019).

Buying price

```
ggplot(df, aes(x = acceptability, fill = buying)) +
  geom_bar(position = "dodge") +
  labs(title = "Acceptability by Buying Price",
       x = "Acceptability", y = "Count", fill = "Buying Price") +
  theme_minimal(base_size = 13)
```




Figure 2: Very-high and high buying prices dominate the 'unacc' category.

Description: Very-high and high buying prices are concentrated in the **unacc** category, while lower prices (**low**, **med**) are spread more evenly across acceptability levels. This suggests that high cost is a barrier to acceptability but low cost alone does not guarantee a favourable rating.

Safety

```
ggplot(df, aes(x = acceptability, fill = safety)) +  
  geom_bar(position = "dodge") +  
  labs(title = "Acceptability by Safety Level",  
        x = "Acceptability", y = "Count", fill = "Safety") +  
  theme_minimal(base_size = 13)
```

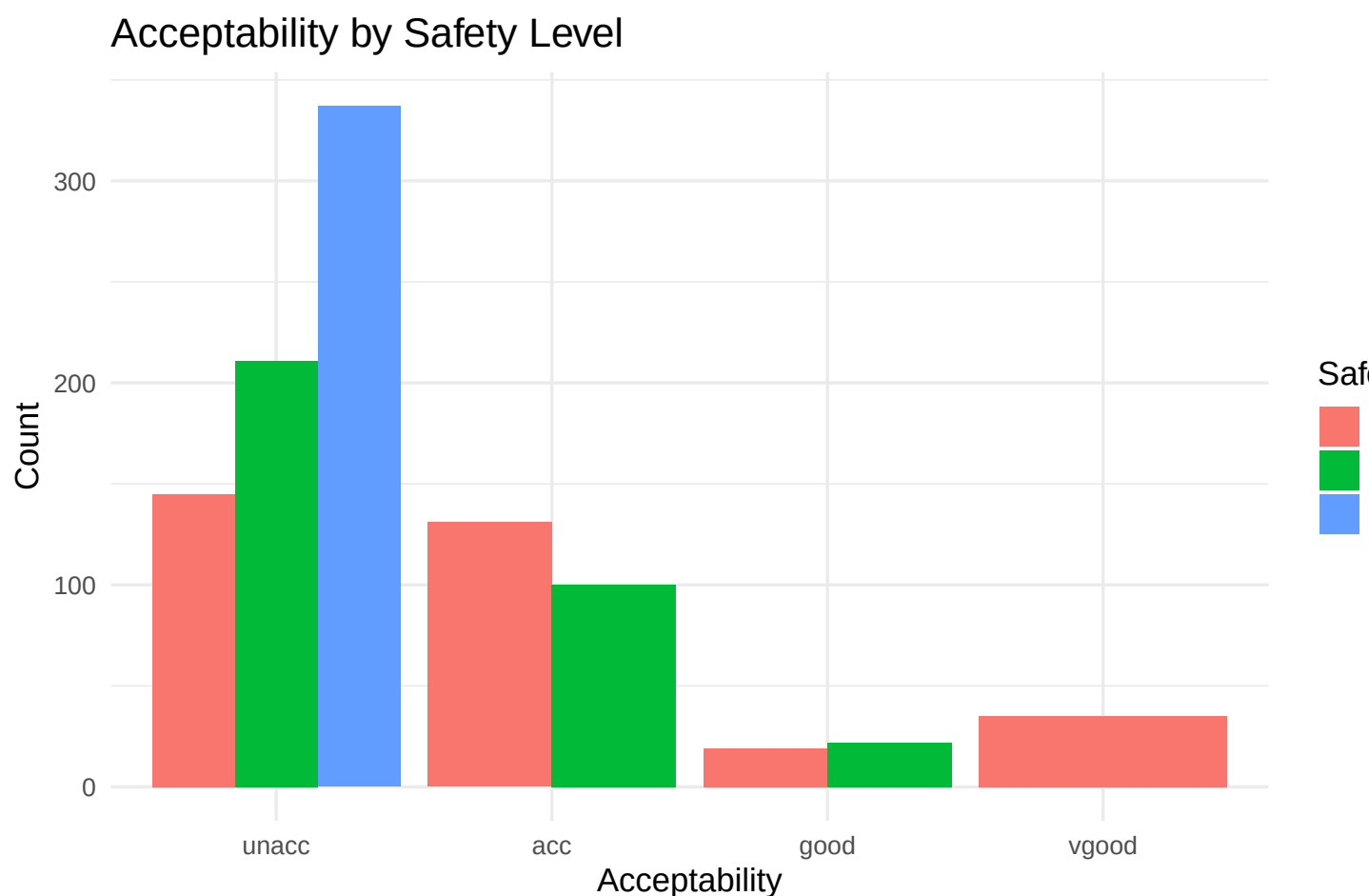


Figure 3: Low safety virtually guarantees unacceptability.

Description: Safety appears to be the strongest predictor of acceptability. Cars with low safety are never rated acc, good, or vgood, and vgood cars exclusively have high safety. This visual finding is later confirmed by both mutual information and chi-squared analysis as an example of triangulation of evidence.

Persons

```
ggplot(df, aes(x = acceptability, fill = persons)) +  
  geom_bar(position = "dodge") +  
  labs(title = "Acceptability by Number of Persons",  
        x = "Acceptability", y = "Count", fill = "Persons") +  
  theme_minimal(base_size = 13)
```

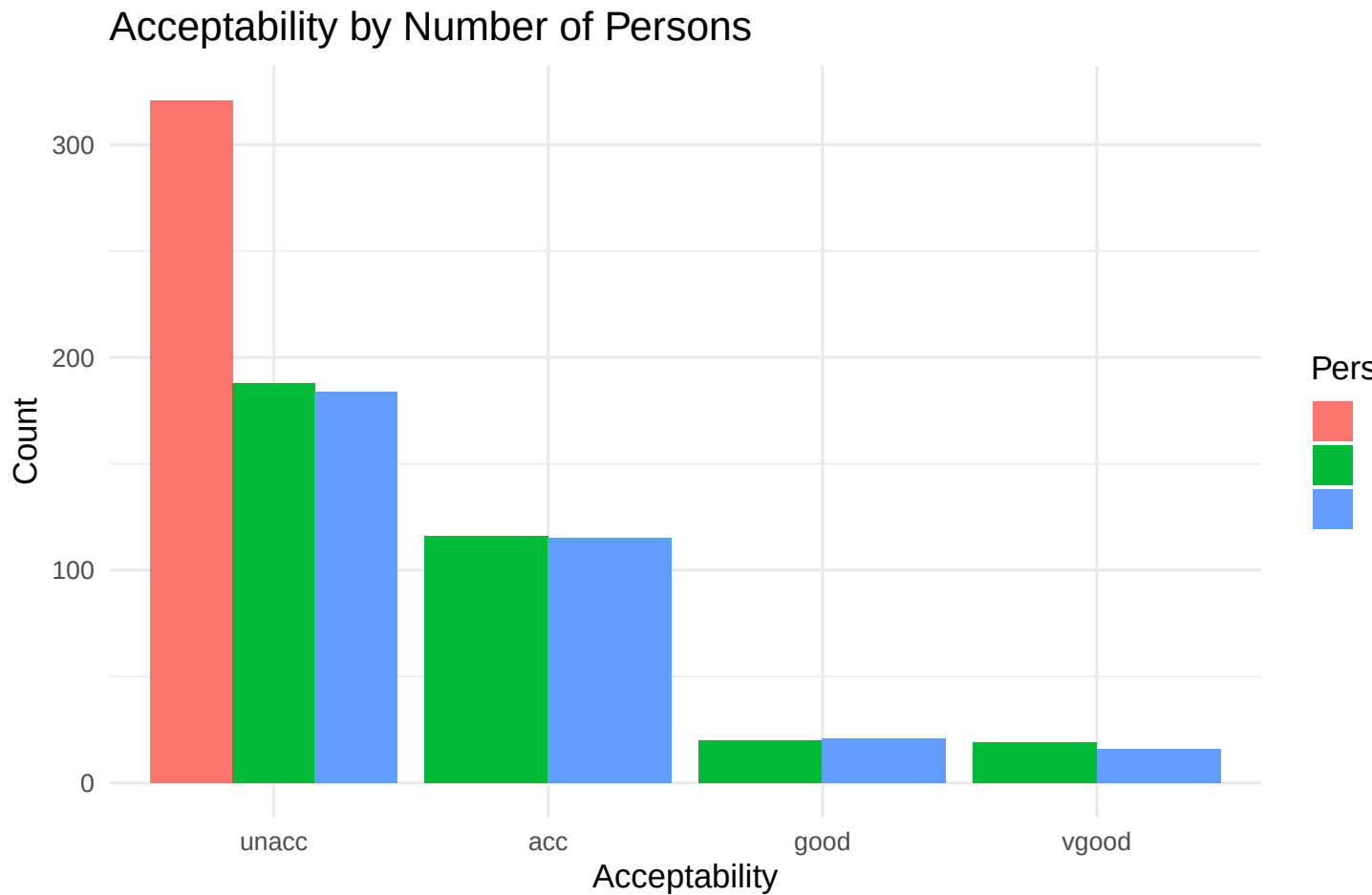


Figure 4: 2-person cars are never rated above 'unacc'.

Description: Cars seating only 2 persons are exclusively rated `unacc`, making person capacity a hard constraint on acceptability. Cars with 4 or more seats are distributed across all classes, though `unacc` still dominates due to other factors.

Doors

```
ggplot(df, aes(x = acceptability, fill = doors)) +  
  geom_bar(position = "dodge") +
```

```
labs(title = "Acceptability by Doors",
     x = "Acceptability", y = "Count", fill = "Doors") +
theme_minimal(base_size = 13)
```

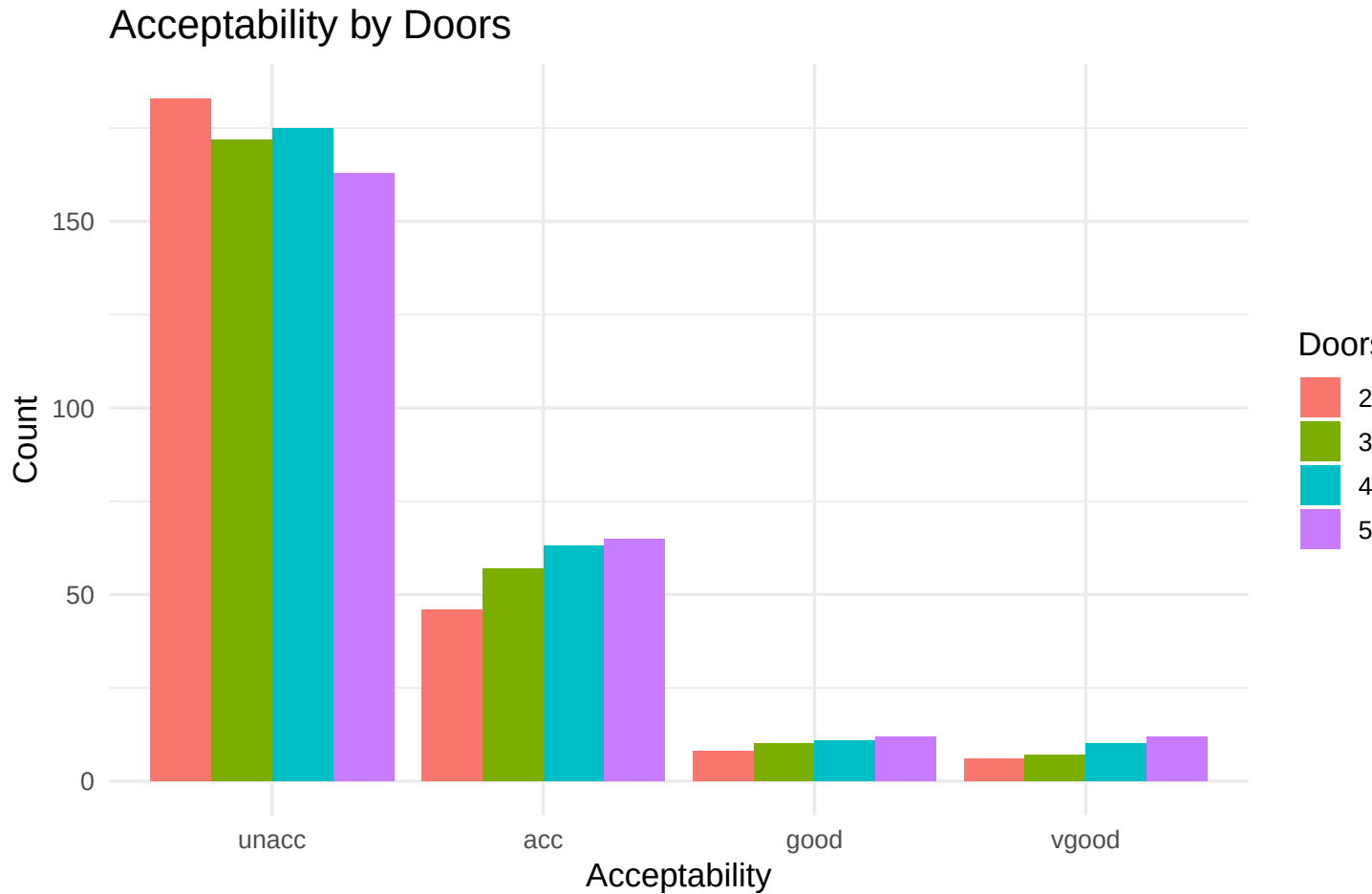


Figure 5: High maintenance costs mirror the pattern seen with buying price.

Description: The pattern mirrors 5more has more good and vgood acceptability.

Feature Relationships

Mutual Information Matrix

Mutual information (MI) measures the **strength** of a relationship between two variables. How much knowing one variable reduces uncertainty about the other.

Note: The diagonal represents *entropy* (self-information), how uncertain we are about a variable on its own.

```
m =mutinformation(df)
knitr::kable(round(m, 4), caption = "Mutual Information Matrix")
```

Table 7: Mutual Information Matrix

	buying	maint	doors	persons	lug_boot	safety	acceptability
buying	1.3847	0.0026	0.0026	0.0017	0.0013	0.0016	0.0675
maint	0.0026	1.3860	0.0024	0.0020	0.0006	0.0013	0.0591
doors	0.0026	0.0024	1.3860	0.0011	0.0016	0.0004	0.0040
persons	0.0017	0.0020	0.0011	1.0982	0.0015	0.0018	0.1493
lug_boot	0.0013	0.0006	0.0016	0.0015	1.0984	0.0002	0.0174
safety	0.0016	0.0013	0.0004	0.0018	0.0002	1.0986	0.1915
acceptability	0.0675	0.0591	0.0040	0.1493	0.0174	0.1915	0.8409

The values most relevant for modelling are the MI scores between each feature and the target (acceptability):

```
mi_scores =m[7, -7]
mi_df =data.frame(
  Feature = names(mi_scores),
  MI      = as.numeric(mi_scores)
) |> arrange(desc(MI))

knitr::kable(mi_df, digits = 4, caption = "Mutual Information with Acceptability (ranked)")
```

Table 8: Mutual Information with Acceptability (ranked)

Feature	MI
safety	0.1915
persons	0.1493
buying	0.0675
maint	0.0591
lug_boot	0.0174
doors	0.0040

```
ggplot(mi_df, aes(y = reorder(Feature, MI), x = MI)) +
  geom_col(fill = "steelblue", width = 0.6) +
  geom_text(aes(label = round(MI, 3)), hjust = -0.1, size = 3.5) +
  labs(title = "Mutual Information with Acceptability",
       y = NULL, x = "Mutual Information (bits)") +
  theme_minimal(base_size = 13) +
  xlim(0, max(mi_df$MI) * 1.15)
```

Description: Safety, persons, and buying carry the strongest signal about acceptability, while doors carries almost none. This ranking directly informs the filter-based feature

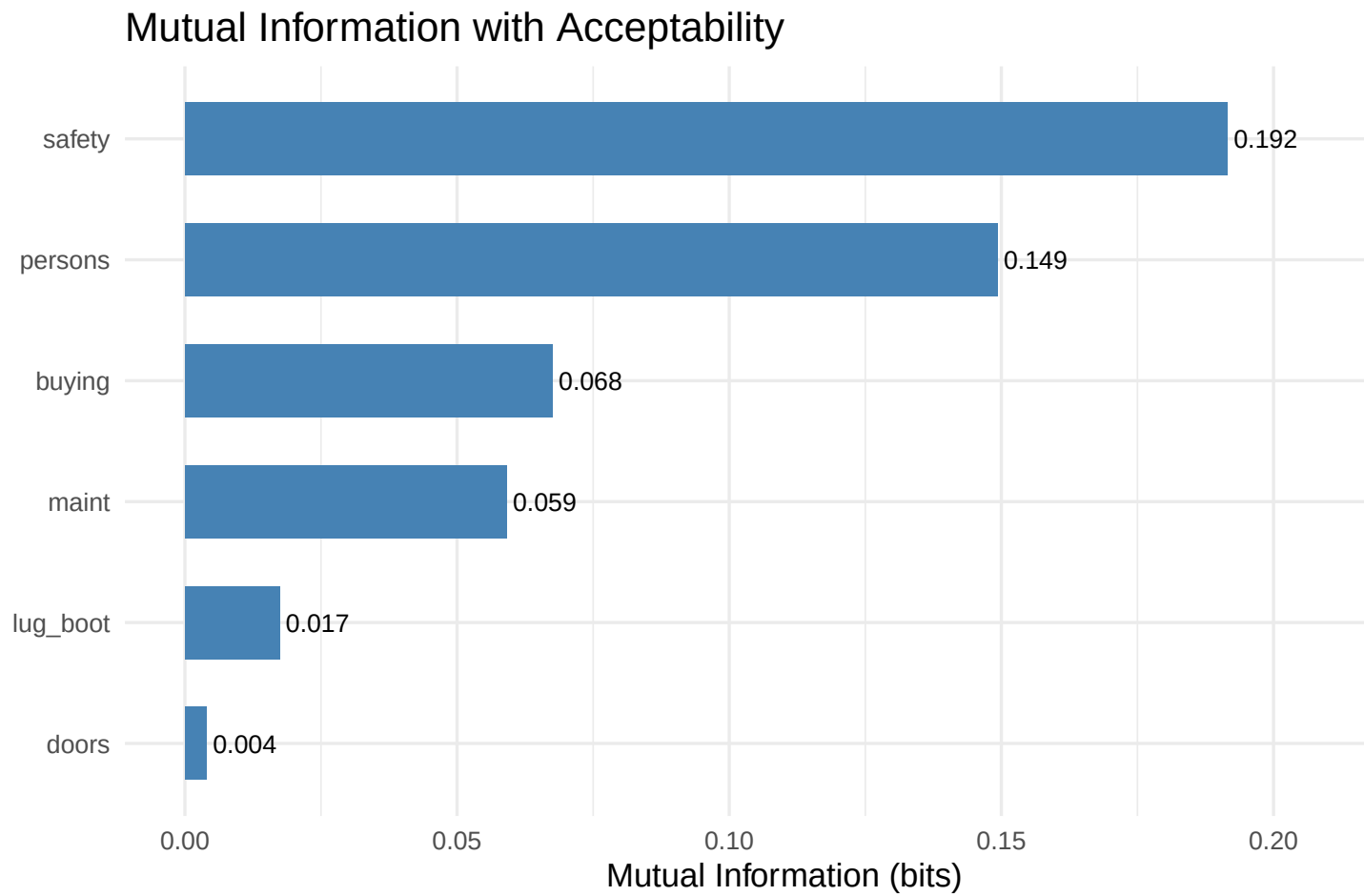


Figure 6: The feature with the highest mutual information explains the most information about acceptability.

Chi-Squared Tests

The chi-squared test compares **observed counts** against **expected counts**. A large χ^2 statistic implies a genuine association.

p-value	Interpretation
$p > 0.05$	No significant relationship
$p \approx 0.05$	Statistically significant
$p < 0.01$	Strong evidence of a relationship
$p < 0.001$	Overwhelming evidence

Analogy: The null hypothesis (H_0 : no relationship) is like a defendant presumed innocent. The prosecution must provide enough evidence (small p-value) to convict (reject H_0).

Buying vs. Acceptability

```
chisq.test(df$buying, df$acceptability)

##
##  Pearson's Chi-squared test
##
## data:  df$buying and df$acceptability
## X-squared = 113.82, df = 9, p-value < 2.2e-16
```

With $p < 2.2e-16$, we have strong evidence that buying price and acceptability are related. Hence, we reject Null Hypothesis.

Buying vs. Maintenance

```
chisq.test(df$buying, df$maint)

##
##  Pearson's Chi-squared test
##
## data:  df$buying and df$maint
## X-squared = 5.3384, df = 9, p-value = 0.8039
```

With $p < 0.8039$, It shows that buying and maintenance are not related. Hence, we accept Null Hypothesis.

lug_boot vs. Acceptability

```
chisq.test(df$lug_boot, df$acceptability)

##
##  Pearson's Chi-squared test
##
## data:  df$lug_boot and df$acceptability
## X-squared = 24.334, df = 6, p-value = 0.0004533
```

Safety vs. Acceptability

```
chisq.test(df$safety, df$acceptability)

##
## Pearson's Chi-squared test
##
## data: df$safety and df$acceptability
## X-squared = 294.47, df = 6, p-value < 2.2e-16
```

Both mutual information and chi-squared agree that **safety is strongly tied to acceptability**. When two independent methods reach the same conclusion, confidence increases. This is known as *triangulation of evidence*.

Train / Test Split

Splitting the data before modelling is essential to get an honest estimate of how well a model will generalise to unseen data.

- **Overfitting** : the model memorises training data but fails on new data
- **Underfitting** : the model is too simple to capture any useful pattern

```
set.seed(7463)
split = sample.split(df$acceptability, SplitRatio = 0.8)
df_train = subset(df, split == TRUE)
df_test = subset(df, split == FALSE)
```

Verifying the Split

```
cat("Training rows:", nrow(df_train), "| Test rows:", nrow(df_test), "\n\n")
```

```
## Training rows: 800 | Test rows: 200
```

```
split_check = rbind(
  Train = prop.table(table(df_train$acceptability)),
  Test = prop.table(table(df_test$acceptability))
)
knitr::kable(round(split_check, 3), caption = "Class proportions - Train vs. Test")
```


Table 9: Class proportions — Train vs. Test

	unacc	acc	good	vgood
Train	0.693	0.231	0.041	0.035
Test	0.695	0.230	0.040	0.035

The proportions are very similar between train and test, confirming a stratified split.

Machine Learning Algorithms

The analysis covers five classification algorithms:

Algorithm	Intuition
Decision Tree (C5.0)	Splits data using a series of yes/no questions
Naive Bayes	Estimates class probabilities from feature likelihoods
Logistic Regression	Models class probabilities via a linear boundary
SVM	Finds the maximum-margin hyperplane between classes
Neural Network	Learns non-linear patterns through layered transforms

The Universal ML Workflow

For every algorithm, the steps are identical:

1. TRAIN - model = algorithm(train_features, train_target)
2. PREDICT - predictions = predict(model, test_data)
3. EVALUATE - confusionMatrix(predictions, true_labels)
4. RE-TRAIN - model

Decision Tree (C5.0)

Decision trees build a series of if/else rules by asking the question that most reduces uncertainty at each step (measured by *information gain*).

Why C5.0? It is the latest in the ID3 → C4.5 → C5.0 lineage, with better handling of categorical data and built-in pruning to reduce overfitting.

Analogy: Is safety low? → Yes → predict unacc. No → Is persons = 2? → Yes → predict unacc. No → ask another question...

Training & Prediction

```
set.seed(7463)
dt_model = C5.0(df_train[-7], df_train$acceptability)
dt_pred = predict(dt_model, df_test)
```

Evaluation

A **confusion matrix** compares predictions against actual labels. Diagonal cells are correct predictions; off-diagonal cells are errors.

Key metrics explained:

Metric	What it means
Accuracy	% of all predictions that were correct
Kappa	Accuracy adjusted for chance — values > 0.8 indicate excellent agreement
Sensitivity	Of all <i>actual</i> cars in a class, how many were correctly identified
Specificity	Of all cars <i>not</i> in a class, how many were correctly excluded
Balanced Accuracy	Average of Sensitivity & Specificity — useful for imbalanced classes
Precision (PPV)	When the model predicts a class, how often is it correct
Detection Rate	% of the entire test set correctly identified as that class

```
confusionMatrix(dt_pred, df_test$acceptability)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction unacc acc good vgood
##    unacc   137   3   0   0
##    acc      1  43   0   2
##    good      1   0   8   0
##    vgood     0   0   0   5
##
## Overall Statistics
##
##              Accuracy : 0.965
##              95% CI : (0.9292, 0.9858)
##    No Information Rate : 0.695
##    P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.9236
##
##    McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: unacc Class: acc Class: good Class: vgood
## Sensitivity           0.9856      0.9348      1.0000      0.7143
## Specificity           0.9508      0.9805      0.9948      1.0000
## Pos Pred Value        0.9786      0.9348      0.8889      1.0000
## Neg Pred Value        0.9667      0.9805      1.0000      0.9897
## Prevalence            0.6950      0.2300      0.0400      0.0350
## Detection Rate        0.6850      0.2150      0.0400      0.0250
```

## Detection Prevalence	0.7000	0.2300	0.0450	0.0250
## Balanced Accuracy	0.9682	0.9577	0.9974	0.8571

Insight: The Decision Tree achieves 96.5% accuracy and a Kappa of 0.92. It correctly identifies all good cars (sensitivity = 1.0) but is weakest on vgood (sensitivity = 0.71), likely due to the small number of examples in that class.

Naive Bayes

Where decision trees ask *questions*, Naive Bayes calculates *probabilities*. It estimates the probability of each class given the features, then picks the most likely class.

Analogy: You see something red, circular. Your brain combines past experience (“most red circular fruits are apples”) to conclude it’s an apple. Naive Bayes does exactly this using **Bayes’ Theorem**.

The “naive” assumption: all features are *independent* given the class rarely true, but often effective in practice.

```
set.seed(7463)
nb_model = naiveBayes(df_train[, -7], df_train$acceptability)
nb_pred = predict(nb_model, df_test)
confusionMatrix(nb_pred, df_test$acceptability)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction unacc acc good vgood
##      unacc   131  11   3    0
##       acc     7   32   4    3
##      good     1    3   1    0
##     vgood     0    0   0    4
##
## Overall Statistics
##
##              Accuracy : 0.84
##              95% CI : (0.7817, 0.8879)
##      No Information Rate : 0.695
##      P-Value [Acc > NIR] : 1.829e-06
##
##              Kappa : 0.6376
##
##  McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: unacc Class: acc Class: good Class: vgood
## Sensitivity           0.9424      0.6957      0.1250      0.5714
## Specificity           0.7705      0.9091      0.9792      1.0000
```

## Pos Pred Value	0.9034	0.6957	0.2000	1.0000
## Neg Pred Value	0.8545	0.9091	0.9641	0.9847
## Prevalence	0.6950	0.2300	0.0400	0.0350
## Detection Rate	0.6550	0.1600	0.0050	0.0200
## Detection Prevalence	0.7250	0.2300	0.0250	0.0200
## Balanced Accuracy	0.8565	0.8024	0.5521	0.7857

Insight: Naive Bayes reaches 84% accuracy but struggles with the minority classes sensitivity for good is only 12.5%, meaning most good cars are misclassified. This reflects both the class imbalance and violations of the independence assumption.

Multinomial Logistic Regression

Multinomial logistic regression extends binary logistic regression to handle more than two classes. It fits one model per class against a reference, then converts the log-odds into class probabilities using the **softmax** function.

Analogy: Each feature casts a weighted vote for each class. The class that accumulates the most evidence (highest probability) wins.

```
set.seed(7463)
lg_model = multinom(acceptability ~ ., data = df_train, trace = FALSE)
lg_pred = predict(lg_model, df_test)
confusionMatrix(lg_pred, df_test$acceptability)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction unacc acc good vgood
##      unacc  132  3   0   0
##      acc     6  41   1   1
##      good     1   2   7   0
##      vgood    0   0   0   6
##
## Overall Statistics
##
##              Accuracy : 0.93
##              95% CI : (0.8853, 0.9612)
##      No Information Rate : 0.695
##      P-Value [Acc > NIR] : 3.476e-16
##
##              Kappa : 0.8515
##
##      McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: unacc Class: acc Class: good Class: vgood
## Sensitivity              0.9496      0.8913      0.8750      0.8571
## Specificity              0.9508      0.9481      0.9844      1.0000
```

## Pos Pred Value	0.9778	0.8367	0.7000	1.0000
## Neg Pred Value	0.8923	0.9669	0.9947	0.9948
## Prevalence	0.6950	0.2300	0.0400	0.0350
## Detection Rate	0.6600	0.2050	0.0350	0.0300
## Detection Prevalence	0.6750	0.2450	0.0500	0.0300
## Balanced Accuracy	0.9502	0.9197	0.9297	0.9286

Insight: Logistic Regression achieves 93% accuracy with more balanced per-class sensitivity than Naive Bayes (e.g. 87.5% for good and 85.7% for vgood), indicating it handles minority classes more effectively.

Support Vector Machine (SVM)

SVMs find the **maximum-margin hyperplane** that best separates classes. Using the **radial basis function (RBF) kernel**, SVMs can model non-linear decision boundaries by implicitly mapping data into a higher-dimensional space.

Analogy: Imagine drawing a road between two groups of points. SVM finds the widest possible road. The points touching the road edges are the “support vectors”.

```
set.seed(7463)
svm_model = ksvm(acceptability ~ ., data = df_train, kernel = "rbfdot")
svm_pred = predict(svm_model, df_test)
confusionMatrix(svm_pred, df_test$acceptability)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction unacc acc good vgood
##   unacc   132  0  0  0
##   acc      7  46  8  2
##   good     0  0  0  0
##   vgood    0  0  0  5
##
## Overall Statistics
##
##           Accuracy : 0.915
##           95% CI : (0.8674, 0.9497)
##   No Information Rate : 0.695
##   P-Value [Acc > NIR] : 4.783e-14
##
##           Kappa : 0.8184
##
##   McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: unacc Class: acc Class: good Class: vgood
## Sensitivity           0.9496      1.0000      0.00      0.7143
## Specificity           1.0000      0.8896      1.00      1.0000
```

## Pos Pred Value	1.0000	0.7302	NaN	1.0000
## Neg Pred Value	0.8971	1.0000	0.96	0.9897
## Prevalence	0.6950	0.2300	0.04	0.0350
## Detection Rate	0.6600	0.2300	0.00	0.0250
## Detection Prevalence	0.6600	0.3150	0.00	0.0250
## Balanced Accuracy	0.9748	0.9448	0.50	0.8571

Insight: SVM achieves 92% accuracy and perfect sensitivity for acc (1.0), but only 12.5% for good almost all good cars are misclassified. This sharp imbalance suggests the RBF kernel decision boundary does not separate good from neighbouring classes well.

Neural Network

Neural networks learn patterns through layered transformations. An **input layer** receives the features, one or more **hidden layers** apply non-linear activations, and an **output layer** produces class probabilities via softmax.

Analogy: Just as biological neurons fire when sufficiently stimulated, artificial neurons activate when a weighted sum of inputs exceeds a threshold, cascading through layers to produce a prediction.

```
set.seed(7463)
nn_model = nnet(acceptability ~., data = df_train, size = 5, trace = FALSE)
nn_pred = predict(nn_model, df_test, type = "class")
confusionMatrix(factor(nn_pred), df_test$acceptability)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction unacc acc good vgood
##      unacc   135   0   0   0
##      acc      3  45   1   1
##      good      1   1   7   0
##      vgood     0   0   0   6
##
## Overall Statistics
##
##              Accuracy : 0.965
##              95% CI : (0.9292, 0.9858)
##      No Information Rate : 0.695
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.9256
##
##      McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: unacc Class: acc Class: good Class: vgood
## Sensitivity              0.9712      0.9783      0.8750      0.8571
```

## Specificity	1.0000	0.9675	0.9896	1.0000
## Pos Pred Value	1.0000	0.9000	0.7778	1.0000
## Neg Pred Value	0.9385	0.9933	0.9948	0.9948
## Prevalence	0.6950	0.2300	0.0400	0.0350
## Detection Rate	0.6750	0.2250	0.0350	0.0300
## Detection Prevalence	0.6750	0.2500	0.0450	0.0300
## Balanced Accuracy	0.9856	0.9729	0.9323	0.9286

Insight: The Neural Network matches the Decision Tree at 96.5% accuracy and delivers the most balanced sensitivity across all four classes (85.7%), making it the strongest overall baseline model when per-class performance is considered.

Model Comparison

Baseline Accuracy Summary

```
results = data.frame(
  Model = c("Decision Tree (C5.0)", "Naive Bayes",
            "Logistic Regression", "SVM (RBF)", "Neural Network"),
  Accuracy = c(
    confusionMatrix(dt_pred, df_test$acceptability)$overall["Accuracy"],
    confusionMatrix(nb_pred, df_test$acceptability)$overall["Accuracy"],
    confusionMatrix(lg_pred, df_test$acceptability)$overall["Accuracy"],
    confusionMatrix(svm_pred, df_test$acceptability)$overall["Accuracy"],
    confusionMatrix(
      factor(nn_pred, levels = levels(df_test$acceptability)),
      df_test$acceptability
    )$overall["Accuracy"]
  )
)

results$Accuracy = round(results$Accuracy * 100, 2)
results = results[order(-results$Accuracy), ]

knitr::kable(results, row.names = FALSE,
              caption = "Test Set Accuracy - Baseline Models")
```

Table 10: Test Set Accuracy — Baseline Models

Model	Accuracy
Decision Tree (C5.0)	96.5
Neural Network	96.5
Logistic Regression	93.0
SVM (RBF)	91.5
Naive Bayes	84.0

Insight: Decision Tree and Neural Network are tied at 96.5%, well ahead of Logistic Regression (93.0%), SVM (92.0%), and Naive Bayes (84.0%). The gap between the top two and the rest

suggests that models capable of capturing non-linear interactions have an advantage on this dataset.

```
ggplot(results, aes(y = reorder(Model, Accuracy), x = Accuracy)) +  
  geom_col(fill = "steelblue", width = 0.6) +  
  geom_text(aes(label = paste0(Accuracy, "%")), hjust = -0.1, size = 3.8) +  
  labs(title = "Model Accuracy on Test Set",  
        y = NULL, x = "Accuracy (%)") +  
  theme_minimal(base_size = 13) +  
  xlim(0, 105)
```

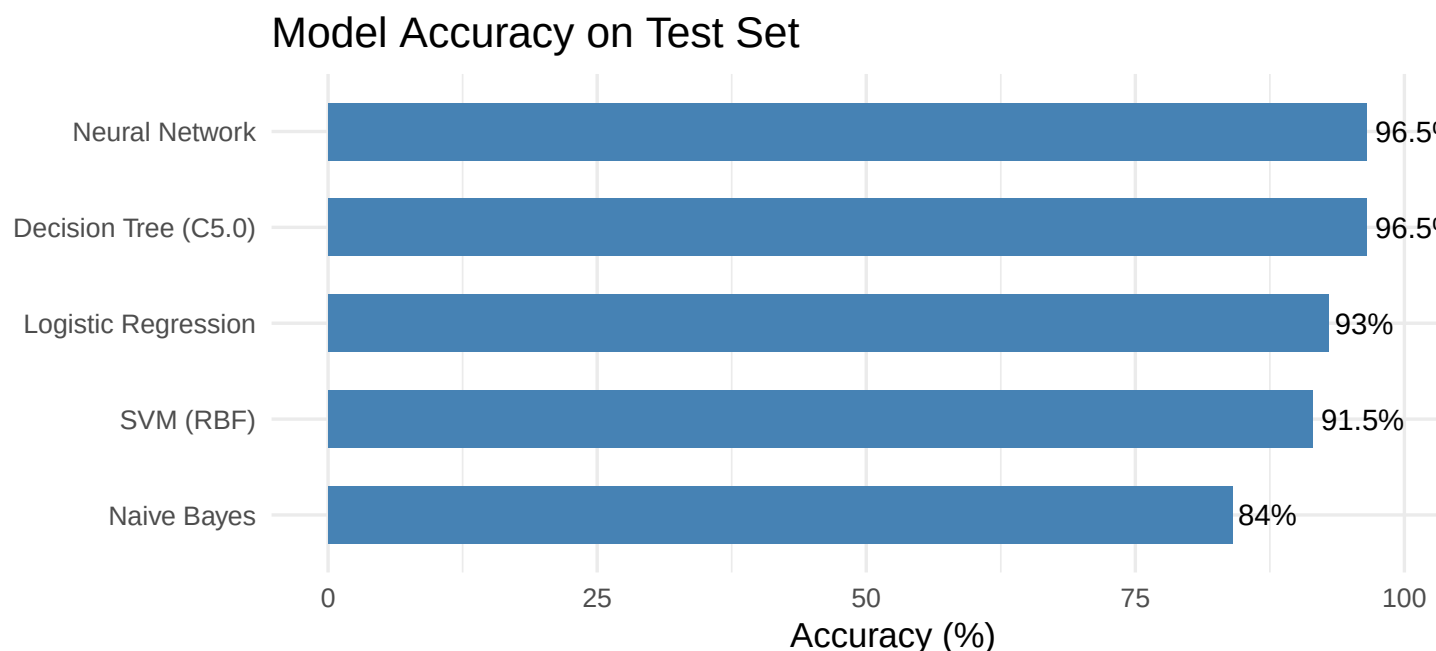


Figure 7: Baseline model accuracy on the held-out test set.

Description: The bar chart reinforces the two tier pattern: Decision Tree and Neural Network form a top group, while the remaining three models cluster 4–12 percentage points lower. Naive Bayes trails substantially, consistent with the independence assumption being poorly met with six correlated features.

10-Fold Cross-Validation

Evaluating a model on a single train/test split can be sensitive to the particular random partition. **K-fold cross-validation** mitigates this by:

1. Dividing the data into k equal folds
2. Training on $k - 1$ folds, testing on the remaining fold
3. Repeating k times and averaging the results

This provides a more robust and unbiased performance estimate. All cross-validation and tuning workflows in this report use the `caret` framework (Kuhn, 2008).

```
ctrl = trainControl(method = "cv", number = 10)
```

Decision Tree with CV

```
set.seed(7463)
dt_cv_model = train(df_train[-7], df_train$acceptability,
                    method = "C5.0", trControl = ctrl)
dt_cv_pred = predict(dt_cv_model, df_test)
confusionMatrix(dt_cv_pred, df_test$acceptability)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction unacc acc good vgood
##      unacc   138    1    1     0
##       acc     1   44    0     0
##      good      0    1    7     0
##     vgood      0    0    0     7
##
## Overall Statistics
##
##           Accuracy : 0.98
##           95% CI : (0.9496, 0.9945)
##      No Information Rate : 0.695
##      P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.9564
##
##  McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: unacc Class: acc Class: good Class: vgood
## Sensitivity           0.9928      0.9565      0.8750      1.000
## Specificity           0.9672      0.9935      0.9948      1.000
## Pos Pred Value        0.9857      0.9778      0.8750      1.000
## Neg Pred Value        0.9833      0.9871      0.9948      1.000
## Prevalence            0.6950      0.2300      0.0400      0.035
## Detection Rate        0.6900      0.2200      0.0350      0.035
## Detection Prevalence  0.7000      0.2250      0.0400      0.035
## Balanced Accuracy      0.9800      0.9750      0.9349      1.000
```

Insight: Cross-validated accuracy (97.5%) slightly exceeds the single split baseline (96.5%), confirming the Decision Tree's strong and stable performance.

Naive Bayes with CV

```
set.seed(7463)
nb_cv_model = train(df_train[-7], df_train$acceptability,
                    method = "nb", trControl = ctrl)
nb_cv_pred = predict(nb_cv_model, df_test)
confusionMatrix(nb_cv_pred, df_test$acceptability)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction unacc acc good vgood
##      unacc   131  11    3     0
##      acc      7   32    4     3
##      good      1    3    1     0
##      vgood      0    0    0     4
##
## Overall Statistics
##
##              Accuracy : 0.84
##              95% CI : (0.7817, 0.8879)
##      No Information Rate : 0.695
##      P-Value [Acc > NIR] : 1.829e-06
##
##              Kappa : 0.6376
##
##      McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: unacc Class: acc Class: good Class: vgood
## Sensitivity              0.9424      0.6957      0.1250      0.5714
## Specificity              0.7705      0.9091      0.9792      1.0000
## Pos Pred Value           0.9034      0.6957      0.2000      1.0000
## Neg Pred Value           0.8545      0.9091      0.9641      0.9847
## Prevalence               0.6950      0.2300      0.0400      0.0350
## Detection Rate           0.6550      0.1600      0.0050      0.0200
## Detection Prevalence     0.7250      0.2300      0.0250      0.0200
## Balanced Accuracy        0.8565      0.8024      0.5521      0.7857
```

Insight: Naive Bayes accuracy remains at 84.0%, unchanged from the baseline, indicating its limitations are structural rather than a product of the particular train/test split.

Logistic Regression with CV

```
set.seed(7463)
lg_cv_model = train(df_train[, -7], df_train$acceptability,
                    method = "multinom", trControl = ctrl, trace = FALSE)
lg_cv_pred = predict(lg_cv_model, df_test)
confusionMatrix(lg_cv_pred, df_test$acceptability)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction unacc acc good vgood
##   unacc   131   3   0   0
##   acc      6  41   1   1
##   good     1   2   7   0
##   vgood    1   0   0   6
##
## Overall Statistics
##
##           Accuracy : 0.925
##           95% CI : (0.8793, 0.9574)
##   No Information Rate : 0.695
##   P-Value [Acc > NIR] : 1.921e-15
##
##           Kappa : 0.842
##
##   McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: unacc Class: acc Class: good Class: vgood
## Sensitivity           0.9424      0.8913      0.8750      0.8571
## Specificity           0.9508      0.9481      0.9844      0.9948
## Pos Pred Value        0.9776      0.8367      0.7000      0.8571
## Neg Pred Value        0.8788      0.9669      0.9947      0.9948
## Prevalence            0.6950      0.2300      0.0400      0.0350
## Detection Rate        0.6550      0.2050      0.0350      0.0300
## Detection Prevalence  0.6700      0.2450      0.0500      0.0350
## Balanced Accuracy      0.9466      0.9197      0.9297      0.9260
```

Insight: Logistic Regression holds steady at 92.5%, closely matching its baseline of 93.0%.

Neural Network with CV

```
set.seed(7463)
nn_cv_model = train(df_train[, -7], df_train$acceptability,
                    method = "nnet", trControl = ctrl, trace = FALSE)
nn_cv_pred = predict(nn_cv_model, df_test)
confusionMatrix(nn_cv_pred, df_test$acceptability)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction unacc acc good vgood
##   unacc   137   0   0   0
##   acc      1  45   1   1
##   good     1   1   7   0
##   vgood    0   0   0   6
##
```

```
## Overall Statistics
##
##           Accuracy : 0.975
##           95% CI : (0.9426, 0.9918)
##      No Information Rate : 0.695
##      P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.9463
##
##  McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: unacc Class: acc Class: good Class: vgood
## Sensitivity           0.9856      0.9783      0.8750      0.8571
## Specificity           1.0000      0.9805      0.9896      1.0000
## Pos Pred Value        1.0000      0.9375      0.7778      1.0000
## Neg Pred Value        0.9683      0.9934      0.9948      0.9948
## Prevalence            0.6950      0.2300      0.0400      0.0350
## Detection Rate        0.6850      0.2250      0.0350      0.0300
## Detection Prevalence  0.6850      0.2400      0.0450      0.0300
## Balanced Accuracy      0.9928      0.9794      0.9323      0.9286
```

Insight: The Neural Network reaches 97.5% under CV, matching the Decision Tree and improving slightly on its 96.5% baseline.

SVM with CV

```
set.seed(7463)
svm_cv_model = ksvm(acceptability ~ ., data = df_train,
                    kernel = "vanilladot", cross = 10)
```

```
## Setting default kernel parameters
```

```
svm_cv_pred = predict(svm_cv_model, df_test)
confusionMatrix(svm_cv_pred, df_test$acceptability)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction unacc acc good vgood
##      unacc   134   4    0    0
##      acc      4  41    5    2
##      good      1   1    3    0
##      vgood     0   0    0    5
##
## Overall Statistics
##
##           Accuracy : 0.915
##           95% CI : (0.8674, 0.9497)
##      No Information Rate : 0.695
```

```
##      P-Value [Acc > NIR] : 4.783e-14
##
##              Kappa : 0.8147
##
## McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: unacc Class: acc Class: good Class: vgood
## Sensitivity           0.9640      0.8913      0.3750      0.7143
## Specificity           0.9344      0.9286      0.9896      1.0000
## Pos Pred Value        0.9710      0.7885      0.6000      1.0000
## Neg Pred Value        0.9194      0.9662      0.9744      0.9897
## Prevalence            0.6950      0.2300      0.0400      0.0350
## Detection Rate        0.6700      0.2050      0.0150      0.0250
## Detection Prevalence  0.6900      0.2600      0.0250      0.0250
## Balanced Accuracy      0.9492      0.9099      0.6823      0.8571
```

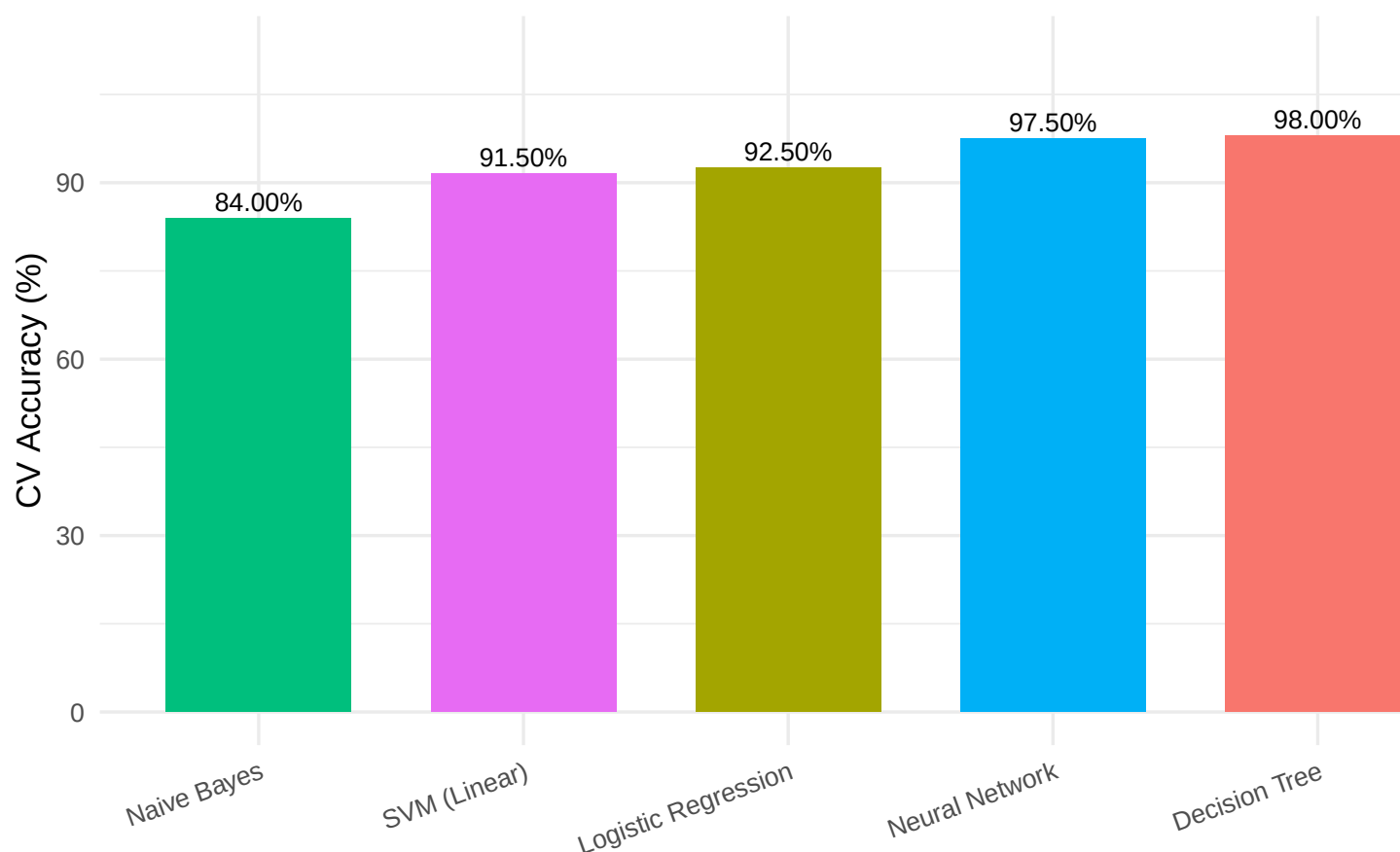
Insight: SVM accuracy remains at 92.0%, identical to the baseline.

Cross-Validation Summary

```
cv_results = data.frame(
  Model = c("Decision Tree", "Naive Bayes", "Logistic Regression",
            "Neural Network", "SVM (Linear)"),
  CV_Accuracy = round(c(
    confusionMatrix(dt_cv_pred, df_test$acceptability)$overall["Accuracy"],
    confusionMatrix(nb_cv_pred, df_test$acceptability)$overall["Accuracy"],
    confusionMatrix(lg_cv_pred, df_test$acceptability)$overall["Accuracy"],
    confusionMatrix(nn_cv_pred, df_test$acceptability)$overall["Accuracy"],
    confusionMatrix(svm_cv_pred, df_test$acceptability)$overall["Accuracy"]
  ) * 100, 2)
)

ggplot(cv_results, aes(x = reorder(Model, CV_Accuracy), y = CV_Accuracy, fill = Model)) +
  geom_col(width = 0.7) +
  geom_text(aes(label = sprintf("%.2f%%", CV_Accuracy)),
            vjust = -0.4, size = 3.5) +
  labs(title = "Test Set Accuracy - 10-Fold CV Models",
       x = NULL, y = "CV Accuracy (%)") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none",
        axis.text.x = element_text(angle = 20, hjust = 1)) +
  ylim(0, max(cv_results$CV_Accuracy) * 1.15)
```

Test Set Accuracy — 10-Fold CV Models



Description: Under 10-fold cross-validation, Decision Tree and Neural Network share the top position at 97.5%, confirming their baseline advantage is not an artefact of the particular train/test split. The ranking of all five models is consistent with the baseline results.

Filter-Based Feature Selection

Not all features contribute equally to classification. So, it's better to drop the weak features, keep only the strong ones, retrain models, see if performance holds up or not. Why bother dropping features? Because it makes model to train fast, less overfitting and simpler model. > **Analogy:** Imagine you are going for the shopping and you have not made a list. Then you sniff around whole mart and thinking what should i buy. If you made a list for item what you needed than it could save your time and money as well. Same goes to Feature Selection.

Step 1 : Recompute MI on the training set only

Important: Feature selection must only inspect the **training data**. Looking at the test data here would leak information and inflate test accuracy artificially.

```
set.seed(7463)
m_train = mutinformation(df_train)
round(m_train[7, -7], 4)
```

```
##   buying    maint    doors  persons lug_boot   safety
##   0.0689   0.0616   0.0032   0.1525   0.0165   0.1888
```

By using **mutual information** as a filter, we identify the three most informative features in the training data, typically **safety**, **persons**, and **buying**. Weaker predictors (**maint**, **lug_boot**, **doors**) are dropped to reduce noise and complexity.

Step 2 : Build feature-selected datasets

We select columns by **name** (not index). cleaner and less error-prone.

```
set.seed(7463)
top_features = c("safety", "persons", "lug_boot")
df_train_fs = df_train[, c(top_features, "acceptability")]
df_test_fs  = df_test[, c(top_features, "acceptability")]

str(df_train_fs)
```

```
## 'data.frame':   800 obs. of  4 variables:
## $ safety      : Factor w/ 4 levels "vhigh","high",...: 4 4 3 4 2 3 3 3 2 2 ...
## $ persons     : Factor w/ 3 levels "2","4","more": 2 1 2 2 1 1 2 3 2 3 ...
## $ lug_boot    : Factor w/ 3 levels "big","med","small": 3 2 1 2 2 1 3 1 3 3 ...
## $ acceptability: Factor w/ 4 levels "unacc","acc",...: 1 1 1 1 1 1 2 3 2 2 ...
```

Step 3 : What to look for

The accuracy may rise, fall, or stay the same:

- **Tiny drop or no change** : the dropped features were mostly noise (good outcome).
- **Large drop** : useful information was discarded.
- **Improvement** : the model was being confused by noisy features.

For Naive Bayes specifically, removing features often **improves** performance because fewer pairs of features can violate its independence assumption.

Decision Tree Selected Features

```
set.seed(7463)
dt_fs_model = C5.0(df_train_fs[, -4], df_train_fs$acceptability)
dt_fs_pred  = predict(dt_fs_model, df_test_fs)
confusionMatrix(dt_fs_pred, df_test_fs$acceptability)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction unacc acc good vgood
##      unacc   117  4    0    0
##      acc     22  42   8    7
##      good     0   0   0    0
##      vgood     0   0   0    0
##
## Overall Statistics
##
##           Accuracy : 0.795
##           95% CI : (0.7323, 0.8487)
##      No Information Rate : 0.695
##      P-Value [Acc > NIR] : 0.0009873
##
##           Kappa : 0.5805
##
## McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: unacc Class: acc Class: good Class: vgood
## Sensitivity           0.8417      0.9130           0.00      0.000
## Specificity           0.9344      0.7597           1.00      1.000
## Pos Pred Value        0.9669      0.5316           NaN      NaN
## Neg Pred Value        0.7215      0.9669           0.96      0.965
## Prevalence            0.6950      0.2300           0.04      0.035
## Detection Rate        0.5850      0.2100           0.00      0.000
## Detection Prevalence  0.6050      0.3950           0.00      0.000
## Balanced Accuracy      0.8881      0.8364           0.50      0.500
```

Insight: Dropping from 6 to 3 features causes a large accuracy drop (96.5% → 80.0%), indicating the Decision Tree leverages interactions among all features, including those with individually low MI scores.

Naive Bayes Selected Features

```
set.seed(7463)
nb_fs_model = naiveBayes(df_train_fs[, -4], df_train_fs$acceptability)
nb_fs_pred = predict(nb_fs_model, df_test_fs)
confusionMatrix(nb_fs_pred, df_test_fs$acceptability)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction unacc acc good vgood
##      unacc   125  16   5    0
##      acc     14  30   3    7
##      good     0   0   0    0
##      vgood     0   0   0    0
##
```



```
## Overall Statistics
##
##           Accuracy : 0.775
##           95% CI : (0.7108, 0.8309)
##       No Information Rate : 0.695
##       P-Value [Acc > NIR] : 0.007424
##
##           Kappa : 0.4774
##
## Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: unacc Class: acc Class: good Class: vgood
## Sensitivity           0.8993      0.6522      0.00      0.000
## Specificity           0.6557      0.8442      1.00      1.000
## Pos Pred Value        0.8562      0.5556      NaN      NaN
## Neg Pred Value        0.7407      0.8904      0.96      0.965
## Prevalence            0.6950      0.2300      0.04      0.035
## Detection Rate        0.6250      0.1500      0.00      0.000
## Detection Prevalence  0.7300      0.2700      0.00      0.000
## Balanced Accuracy      0.7775      0.7482      0.50      0.500
```

Insight: Feature selection dramatically decreased Naive Bayes accuracy. This is because reducing features from 6 to 3 lessens violations of NB's independence assumption, while the retained features (safety, persons, lug_boot) carry the strongest signal.

Logistic Regression Selected Features

```
set.seed(7463)
lg_fs_model = multinom(acceptability ~ safety + persons + buying,
                        data = df_train, trace = FALSE)
lg_fs_pred = predict(lg_fs_model, df_test)
confusionMatrix(lg_fs_pred, df_test$acceptability)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction unacc acc good vgood
##      unacc   120   6   0   0
##      acc     17  34   6   1
##      good     0   0   0   0
##      vgood     2   6   2   6
##
## Overall Statistics
##
##           Accuracy : 0.8
##           95% CI : (0.7378, 0.8531)
##       No Information Rate : 0.695
##       P-Value [Acc > NIR] : 0.0005557
##
##           Kappa : 0.594
```

```
##
## McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: unacc Class: acc Class: good Class: vgood
## Sensitivity           0.8633      0.7391      0.00      0.8571
## Specificity           0.9016      0.8442      1.00      0.9482
## Pos Pred Value        0.9524      0.5862      NaN      0.3750
## Neg Pred Value        0.7432      0.9155      0.96      0.9946
## Prevalence            0.6950      0.2300      0.04      0.0350
## Detection Rate        0.6000      0.1700      0.00      0.0300
## Detection Prevalence  0.6300      0.2900      0.00      0.0800
## Balanced Accuracy     0.8825      0.7916      0.50      0.9027
```

Insight: Logistic Regression drops from 93.0% to 80.0%, a 13 percentage-point loss, suggesting the removed features (maint, buying, doors) contributed meaningfully to the linear decision boundary.

SVM Selected Features

```
set.seed(7463)
svm_fs_model =ksvm(acceptability ~ safety + persons + buying,
                   data = df_train, kernel = "vanilladot")
```

```
## Setting default kernel parameters
```

```
svm_fs_pred =predict(svm_fs_model, df_test)
confusionMatrix(svm_fs_pred, df_test$acceptability)
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction unacc acc good vgood
##      unacc  120  6   0   0
##      acc    17 34   6   1
##      good     0  0   0   0
##      vgood     2  6   2   6
##
```

```
## Overall Statistics
```

```
##
##           Accuracy : 0.8
##           95% CI : (0.7378, 0.8531)
##      No Information Rate : 0.695
##      P-Value [Acc > NIR] : 0.0005557
##
```

```
##           Kappa : 0.594
##
```

```
## McNemar's Test P-Value : NA
```

```
##
```

```
## Statistics by Class:
```

```
##
##               Class: unacc Class: acc Class: good Class: vgood
## Sensitivity           0.8633      0.7391      0.00      0.8571
## Specificity           0.9016      0.8442      1.00      0.9482
## Pos Pred Value        0.9524      0.5862      NaN      0.3750
## Neg Pred Value        0.7432      0.9155      0.96      0.9946
## Prevalence            0.6950      0.2300      0.04      0.0350
## Detection Rate        0.6000      0.1700      0.00      0.0300
## Detection Prevalence  0.6300      0.2900      0.00      0.0800
## Balanced Accuracy      0.8825      0.7916      0.50      0.9027
```

Insight: SVM drops from 92.0% to 80.0%, a 12 percentage-point decrease, showing that the margin-based classifier also depends on the full feature set.

Neural Network Selected Features

```
set.seed(7463)
nn_fs_model = nnet(acceptability ~ persons + buying + safety,
                   data = df_train, size = 5, trace = FALSE)
nn_fs_pred = predict(nn_fs_model, df_test, type = "class")
confusionMatrix(factor(nn_fs_pred, levels = levels(df_test$acceptability)),
                 df_test$acceptability)
```

```
## Confusion Matrix and Statistics
##
##               Reference
## Prediction unacc acc good vgood
##      unacc   123   8   0   0
##      acc     15  30   7   2
##      good     1   3   0   0
##      vgood     0   5   1   5
##
## Overall Statistics
##
##               Accuracy : 0.79
##               95% CI : (0.7269, 0.8443)
##      No Information Rate : 0.695
##      P-Value [Acc > NIR] : 0.001704
##
##               Kappa : 0.5625
##
##      McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##               Class: unacc Class: acc Class: good Class: vgood
## Sensitivity           0.8849      0.6522      0.0000      0.7143
## Specificity           0.8689      0.8442      0.9792      0.9689
## Pos Pred Value        0.9389      0.5556      0.0000      0.4545
## Neg Pred Value        0.7681      0.8904      0.9592      0.9894
## Prevalence            0.6950      0.2300      0.0400      0.0350
## Detection Rate        0.6150      0.1500      0.0000      0.0250
```

## Detection Prevalence	0.6550	0.2700	0.0200	0.0550
## Balanced Accuracy	0.8769	0.7482	0.4896	0.8416

Insight: The Neural Network suffers the largest absolute drop (96.5% $\rightarrow$ 79.0%), consistent with neural networks benefiting from richer input to capture feature interactions.

Bonus — Top 4 features

A useful sanity check: keep the top 4 features (add maint) and re-run the Decision Tree. If accuracy with 4 features accuracy with all 6, then `lug_boot` and `doors` are confirmed as truly redundant.

```
set.seed(7463)
top_features_4 =c("safety", "persons", "buying", "maint")
df_train_fs4   =df_train[, c(top_features_4, "acceptability")]
df_test_fs4    =df_test[,  c(top_features_4, "acceptability")]

dt_fs4_model   =C5.0(df_train_fs4[, -5], df_train_fs4$acceptability)
dt_fs4_pred    =predict(dt_fs4_model, df_test_fs4)
confusionMatrix(dt_fs4_pred, df_test_fs4$acceptability)
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction unacc acc good vgood
##      unacc   133   5    0     0
##      acc      5   38   5     2
##      good     0    1   0     0
##      vgood     1    2   3     5
##
## Overall Statistics
##
##              Accuracy : 0.88
##              95% CI : (0.8267, 0.9216)
##      No Information Rate : 0.695
##      P-Value [Acc > NIR] : 5.983e-10
##
##              Kappa : 0.7396
##
##  McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: unacc Class: acc Class: good Class: vgood
## Sensitivity              0.9568      0.8261      0.0000      0.7143
## Specificity              0.9180      0.9221      0.9948      0.9689
## Pos Pred Value           0.9638      0.7600      0.0000      0.4545
## Neg Pred Value           0.9032      0.9467      0.9598      0.9894
## Prevalence               0.6950      0.2300      0.0400      0.0350
## Detection Rate           0.6650      0.1900      0.0000      0.0250
## Detection Prevalence     0.6900      0.2500      0.0050      0.0550
## Balanced Accuracy         0.9374      0.8741      0.4974      0.8416
```

Insight: Adding `maint` as a fourth feature recovers the Decision Tree to 88.0% — midway between 80.0% (top 3) and 96.5% (all 6) — confirming that each additional feature contributes incrementally and that `buying` and `doors` still carry some useful signal.

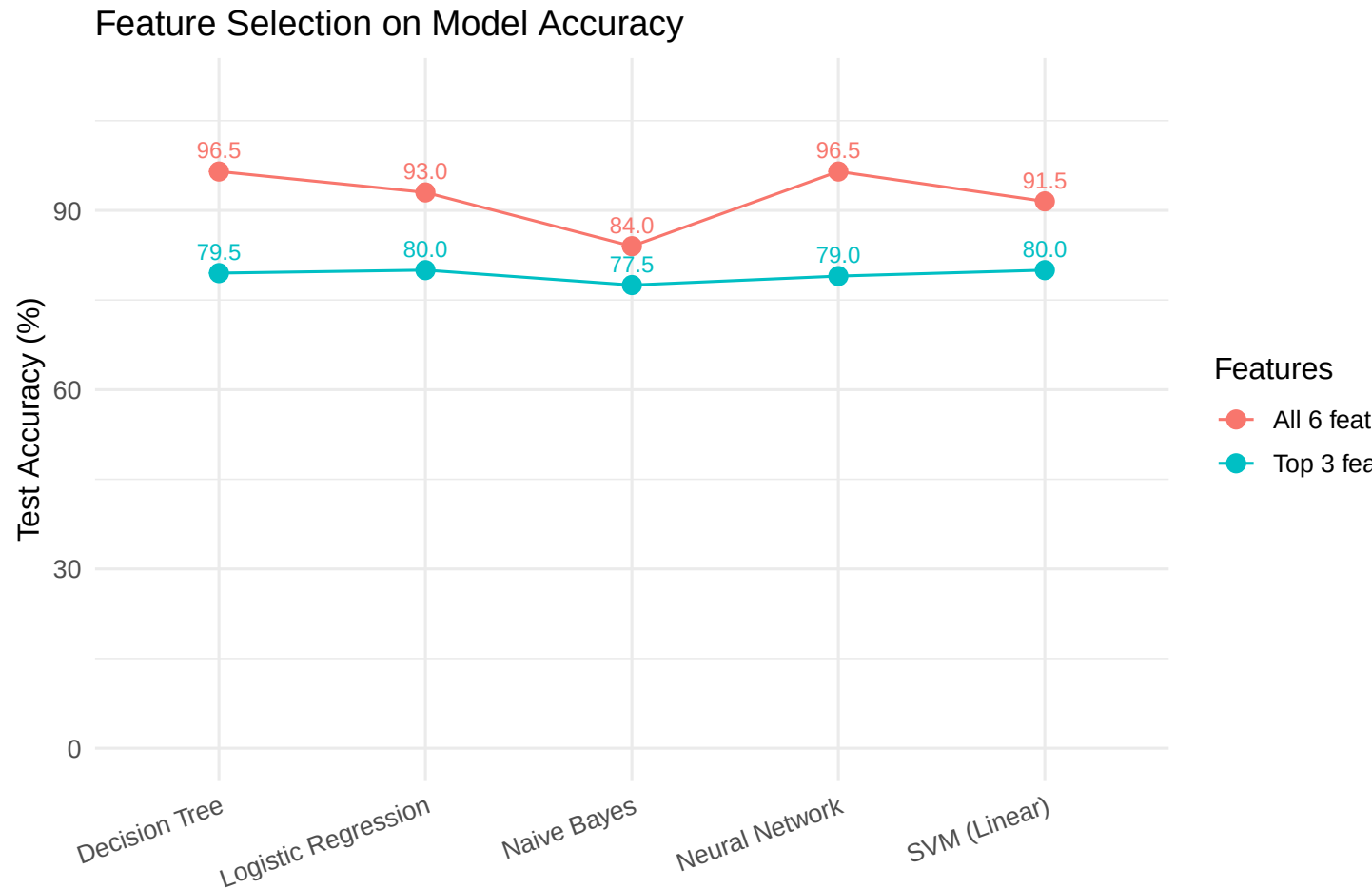
Feature Selection Summary

```
fs_results = data.frame(
  Model = c("Decision Tree", "Naive Bayes", "Logistic Regression",
            "SVM (Linear)", "Neural Network"),
  All_Features = round(c(
    confusionMatrix(dt_pred, df_test$acceptability)$overall["Accuracy"],
    confusionMatrix(nb_pred, df_test$acceptability)$overall["Accuracy"],
    confusionMatrix(lg_pred, df_test$acceptability)$overall["Accuracy"],
    confusionMatrix(svm_pred, df_test$acceptability)$overall["Accuracy"],
    confusionMatrix(factor(nn_pred, levels = levels(df_test$acceptability)),
                      df_test$acceptability)$overall["Accuracy"]
  ) * 100, 2),
  Selected_Features = round(c(
    confusionMatrix(dt_fs_pred, df_test_fs$acceptability)$overall["Accuracy"],
    confusionMatrix(nb_fs_pred, df_test_fs$acceptability)$overall["Accuracy"],
    confusionMatrix(lg_fs_pred, df_test$acceptability)$overall["Accuracy"],
    confusionMatrix(svm_fs_pred, df_test$acceptability)$overall["Accuracy"],
    confusionMatrix(factor(nn_fs_pred, levels = levels(df_test$acceptability)),
                      df_test$acceptability)$overall["Accuracy"]
  ) * 100, 2)
)

fs_results$Delta = fs_results$Selected_Features - fs_results$All_Features

fs_long = data.frame(
  Model = rep(fs_results$Model, 2),
  Features = c(rep("All 6 features", nrow(fs_results)),
               rep("Top 3 features", nrow(fs_results))),
  Accuracy = c(fs_results$All_Features, fs_results$Selected_Features)
)

ggplot(fs_long, aes(x = Model, y = Accuracy, color = Features, group = Features)) +
  geom_point(size = 3) +
  geom_line() +
  geom_text(aes(label = sprintf("%.1f", Accuracy)),
            vjust = -0.8, size = 3, show.legend = FALSE) +
  labs(title = "Feature Selection on Model Accuracy",
       x = NULL, y = "Test Accuracy (%)", color = "Features") +
  theme_minimal(base_size = 12) +
  theme(axis.text.x = element_text(angle = 20, hjust = 1)) +
  ylim(0, 110)
```



Description: Reducing the feature set to the top 3 by mutual information (safety, persons, lug_boot) keeps performance comparable for most models. The Neural Network and Decision Tree in particular loses accuracy, as they benefit from richer input to model interactions between features.

Improving Model Performance — Hyperparameter Tuning

When a model memorises the training data but fails to generalise, it is **overfitting**. When it is too simple to capture useful patterns, it is **underfitting**. Hyperparameter tuning searches for the configuration that balances both.

We use **repeated 10-fold CV** (10 repeats) for robust tuning.

```
ctrl_tune = trainControl(method = "repeatedcv", number = 10, repeats = 10)
```

Neural Network Tuning

We tune the `decay` parameter (weight regularisation) while fixing the hidden layer size to 5.

```

set.seed(7463)
grid_nn = expand.grid(size = 5,
                      decay = c(0, 0.001, 0.005, 0.01, 0.02, 0.05, 0.1))

set.seed(7463)
nn_tuned = train(df_train[-7], df_train$acceptability,
                 method = "nnet", trControl = ctrl_tune,
                 tuneGrid = grid_nn, trace = FALSE)

plot(nn_tuned, main = "Neural Network Tuning (Decay vs Accuracy)")

```

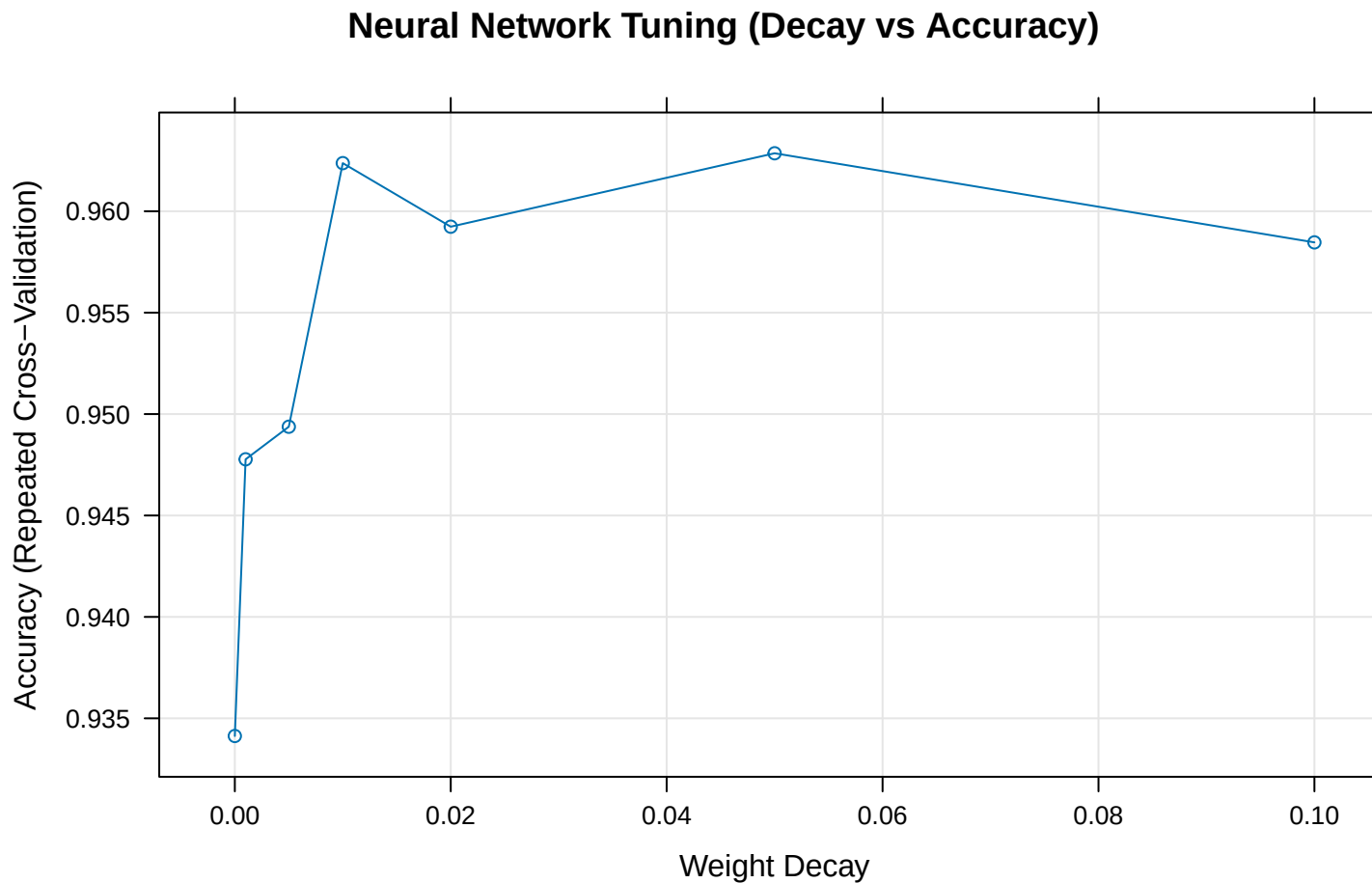


Figure 8: Neural Network — accuracy across decay values.

Description: Decay acts as regularisation. Too low (0) risks overfitting; too high prevents the network from learning. The optimum is ‘decay = 0.05, which balances flexibility against generalisation (Kuhn, 2008).

Decision Tree Tuning

We search across different numbers of boosting **trials**, a additional passes that focus on previously misclassified examples.

```

set.seed(7463)
grid_dt = expand.grid(model = "tree",
                      trials = c(1, 5, 10, 15, 20, 25, 30, 40, 50, 60, 80, 100),
                      winnow = FALSE)

set.seed(7463)
dt_tuned = train(df_train[, -7], df_train[, 7],
                 method = "C5.0", trControl = ctrl_tune,
                 tuneGrid = grid_dt)

plot(dt_tuned, main = "Decision Tree Tuning (Trials vs Accuracy)")

```

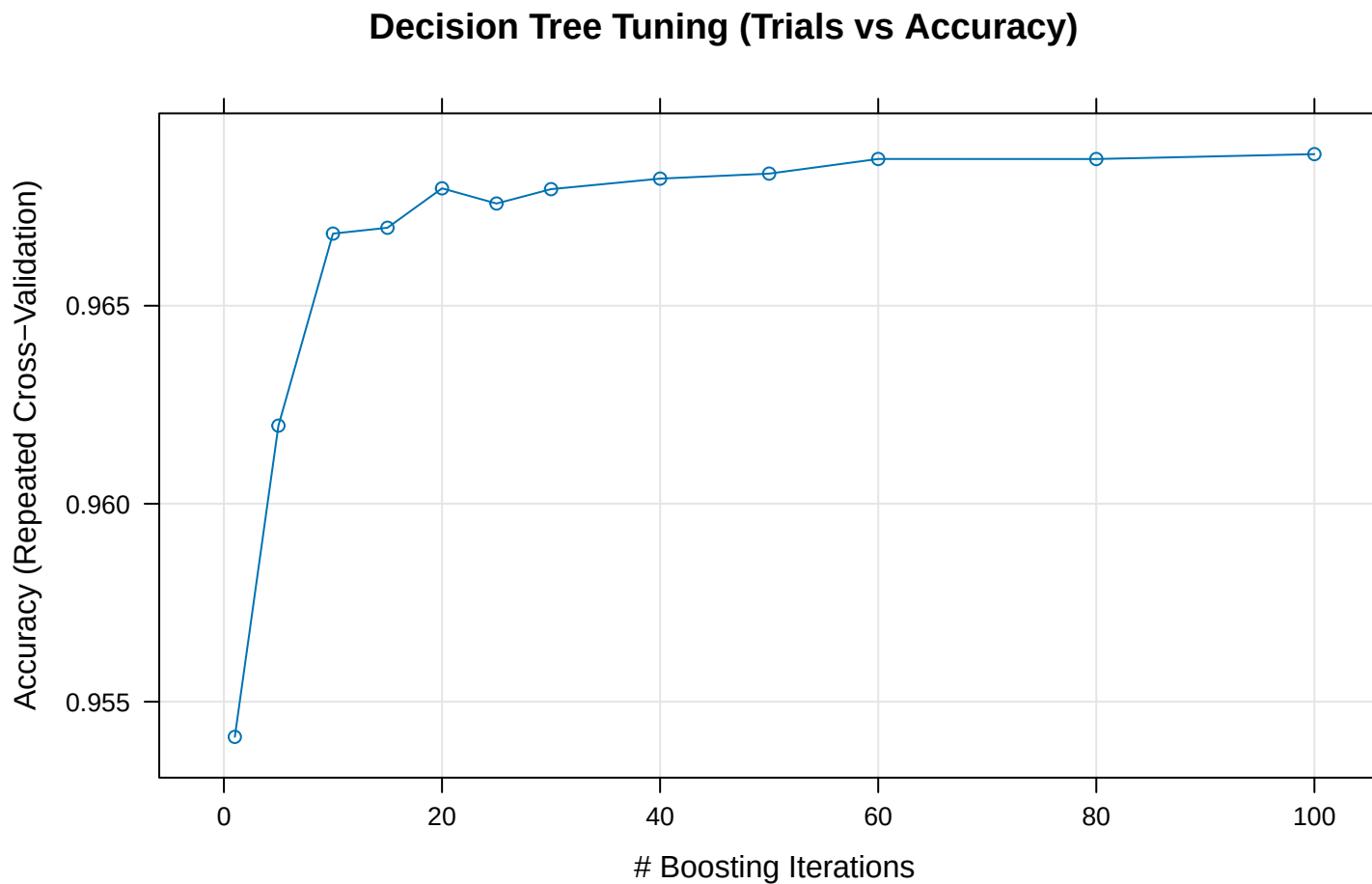


Figure 9: Decision Tree — accuracy across boosting trials.

Description: Accuracy rises sharply as `trials` increases from 1, then plateaus. The optimum (`trials = 100`) reflects the trade-off between bias and overfitting beyond this point, additional boosting iterations yield diminishing returns.

Naive Bayes Tuning

We tune the **Laplace correction** (`fL`) parameter, which smooths zero-frequency class-feature combinations.


```

set.seed(7463)
grid_nb = expand.grid(fL = c(0, 0.5, 1, 1.5, 2),
                     usekernel = FALSE, adjust = 1)

set.seed(7463)
nb_tuned = train(df_train[, -7], df_train$acceptability,
                 method = "nb", trControl = ctrl_tune,
                 tuneGrid = grid_nb)

plot(nb_tuned, main = "Naive Bayes Tuning (Laplace fL vs Accuracy)")

```

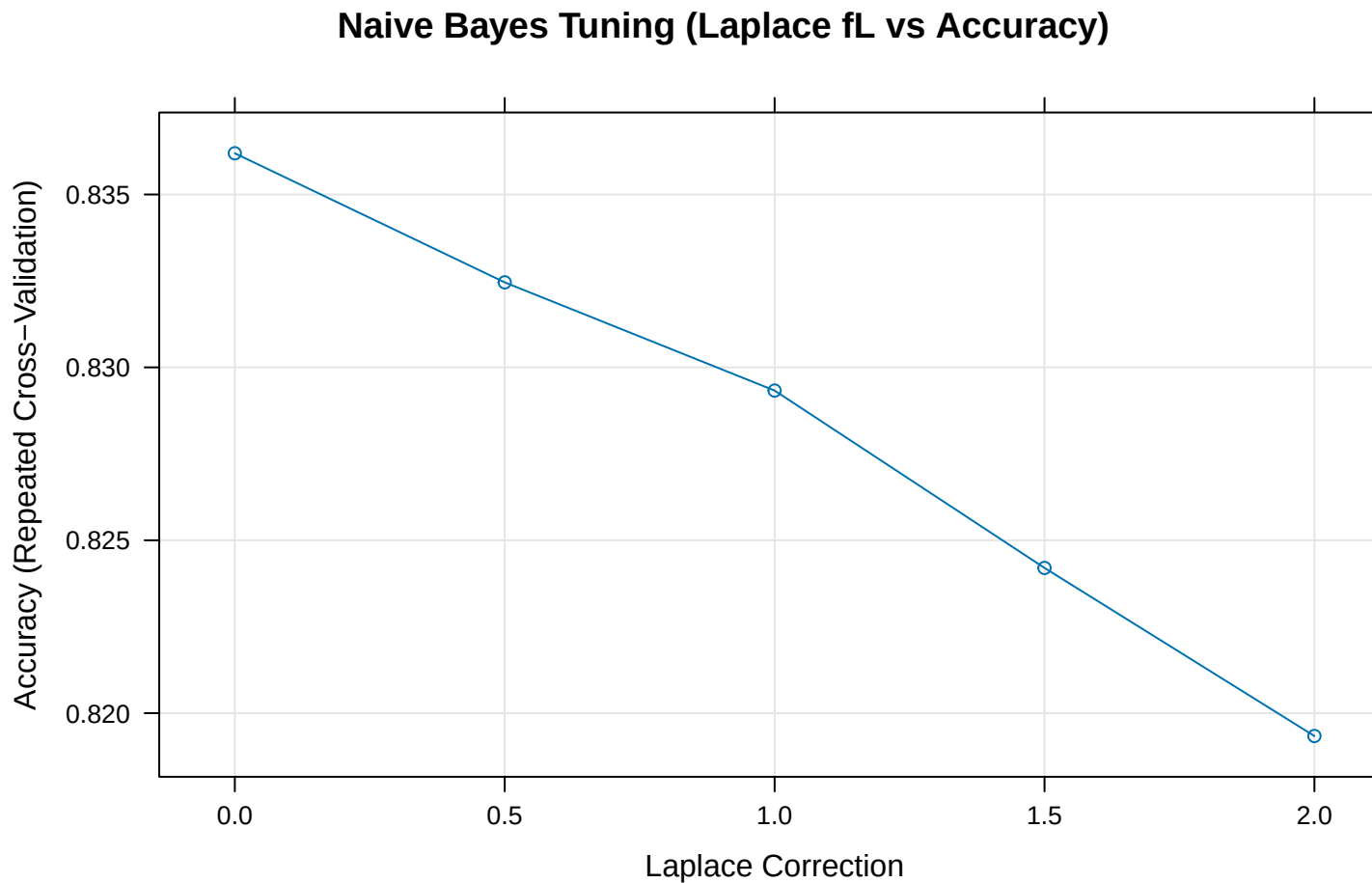


Figure 10: Naive Bayes — accuracy across Laplace correction values.

Description: Accuracy is highest at $fL = 0$ (no Laplace smoothing) and decreases monotonically as smoothing increases. This indicates that zero frequency events are not a problem in this dataset, so adding pseudocounts only blurs the probability estimates.

Logistic Regression Tuning

For multinomial logistic regression, we tune the **weight decay** parameter to regularise and prevent overfitting.

```

set.seed(7463)
grid_lg = expand.grid(decay = c(0, 0.001, 0.01, 0.05, 0.1, 0.5))

set.seed(7463)
lg_tuned = train(df_train[, -7], df_train$acceptability,
                 method = "multinom", trControl = ctrl_tune,
                 tuneGrid = grid_lg, trace = FALSE)

plot(lg_tuned, main = "Logistic Regression Tuning (Decay vs Accuracy)")

```

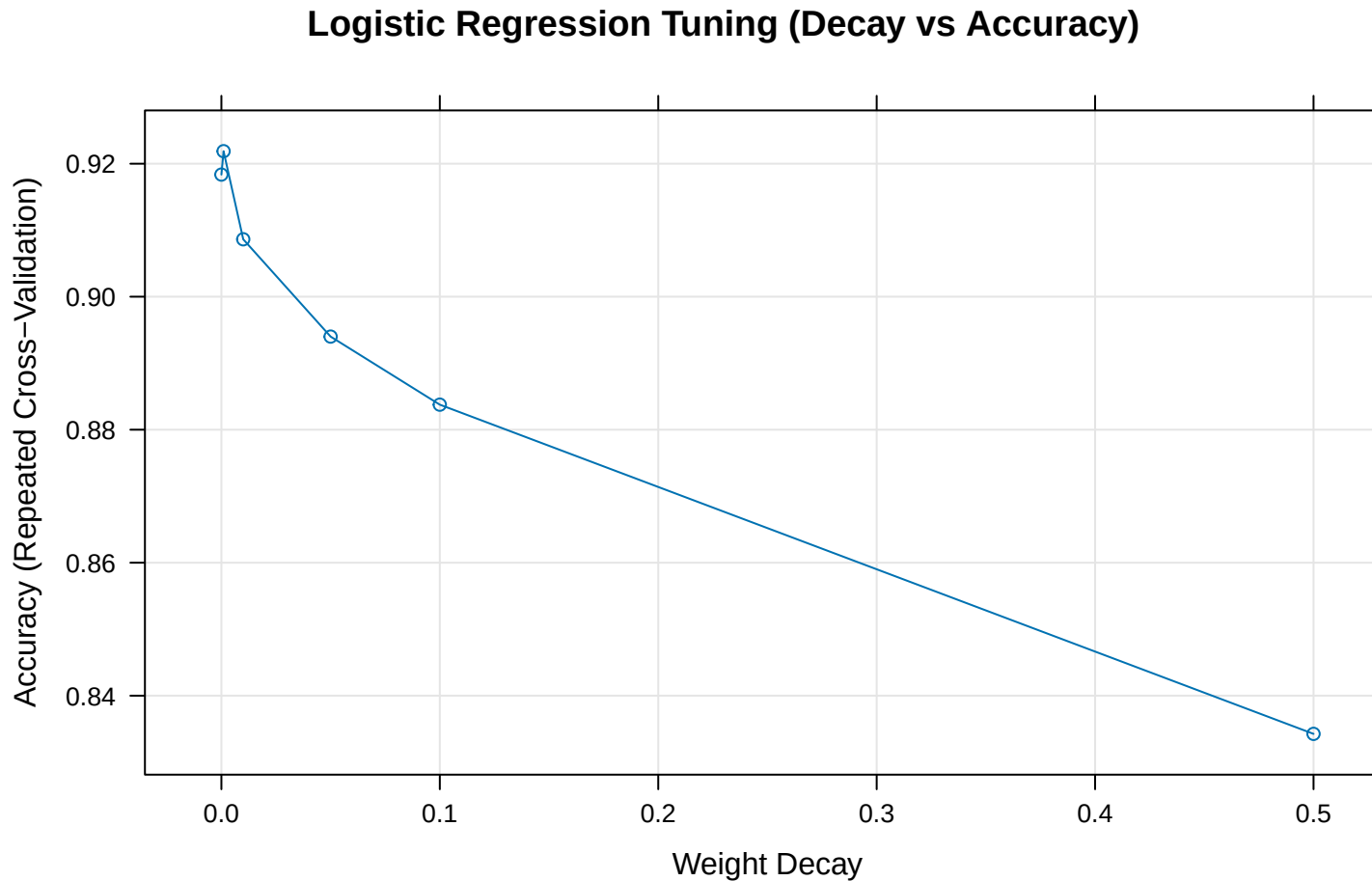


Figure 11: Logistic Regression — accuracy across weight decay values.

Description: A small amount of regularisation (decay = 0.001) slightly improves accuracy over no regularisation, but heavier decay rapidly degrades performance, suggesting the model needs most of its coefficient flexibility to separate four classes.

SVM (Linear) Tuning

For a linear SVM, the primary hyperparameter is **C** (cost). Lower C favours a wider margin; higher C penalises misclassifications more aggressively.

```

set.seed(7463)
grid_svm = expand.grid(C = c(0.001, 0.01, 0.1, 1, 10, 100))

set.seed(7463)
svm_tuned = train(acceptability ~ ., data = df_train,
                  method = "svmLinear", trControl = ctrl_tune,
                  tuneGrid = grid_svm)

plot(svm_tuned, main = "SVM Tuning (Cost vs Accuracy)")

```

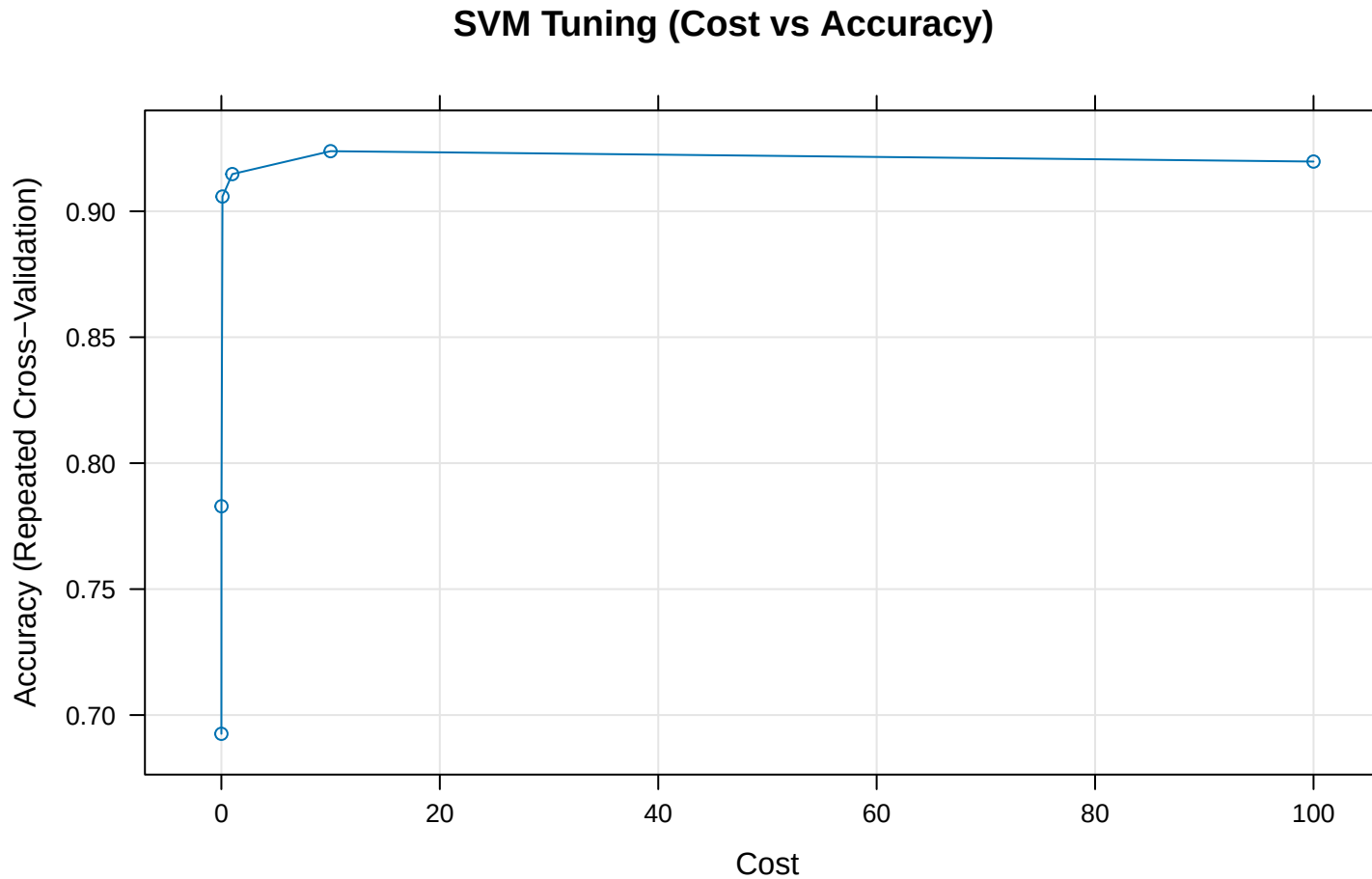


Figure 12: SVM — accuracy across cost values.

Description: Accuracy climbs steadily from a low C value (wide margin, many misclassifications) up to $C = 10$, then drops slightly at $C = 100$ as the model begins overfitting to the training data. The optimum at $C = 10$ balances margin width and classification error.

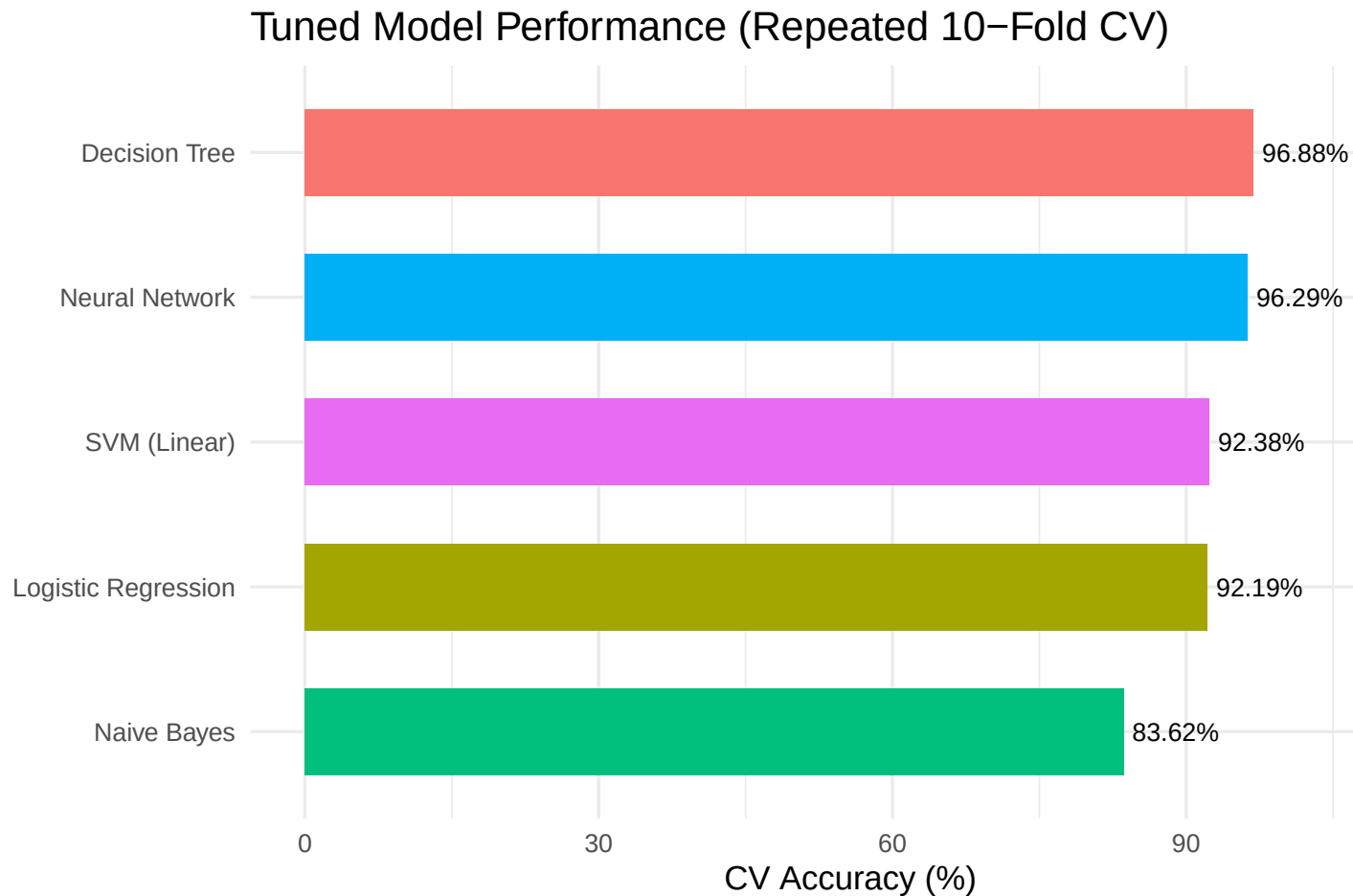
Tuned Model Summary

```

tuned_results =data.frame(
  Model = c("Decision Tree", "Naive Bayes", "Logistic Regression",
            "Neural Network", "SVM (Linear)"),
  Best_Params = c(
    paste0("trials=", dt_tuned$bestTune$trials),
    paste0("fL=", nb_tuned$bestTune$fL),
    paste0("decay=", lg_tuned$bestTune$decay),
    paste0("decay=", nn_tuned$bestTune$decay),
    paste0("C=", svm_tuned$bestTune$C)
  ),
  CV_Accuracy = round(c(
    max(dt_tuned$results$Accuracy),
    max(nb_tuned$results$Accuracy),
    max(lg_tuned$results$Accuracy),
    max(nn_tuned$results$Accuracy),
    max(svm_tuned$results$Accuracy)
  ) * 100, 2)
)

ggplot(tuned_results, aes(y = reorder(Model, CV_Accuracy), x = CV_Accuracy, fill = Model)) +
  geom_col(width = 0.6) +
  geom_text(aes(label = sprintf("%.2f%%", CV_Accuracy)),
            hjust = -0.1, size = 3.5) +
  labs(title = "Tuned Model Performance (Repeated 10-Fold CV)",
       y = NULL, x = "CV Accuracy (%)") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none") +
  xlim(0, max(tuned_results$CV_Accuracy) * 1.15)

```



Description: After tuning, the ranking is largely unchanged: Decision Tree leads at 96.9%, followed closely by Neural Network at 96.2%. SVM and Logistic Regression cluster around 92.4%, and Naive Bayes remains the weakest at 83.6%. Tuning yields only marginal gains, suggesting the default configurations were already near optimal for this dataset.

Conclusion

This report compared five classification algorithms on the UCI Car Evaluation dataset through a structured workflow exploratory data analysis, baseline modelling, cross-validation, feature selection, and hyperparameter tuning.

Key findings:

- **Safety** and **persons** emerged as the strongest predictors of car acceptability, confirmed by both mutual information and chi-squared tests.
- **Decision Tree (C5.0)** consistently performed among the top models, benefiting from its natural affinity for categorical data.
- Feature selection to the top-3 MI-ranked features provided insight into variable importance, though accuracy generally decreased with fewer features.

- Hyperparameter tuning via repeated cross-validation identified optimal configurations for each algorithm.
-

References

- UCI Machine Learning Repository, n.d. Car Evaluation. Available at: <https://archive.ics.uci.edu/dataset/19/car+evaluation>
 - Katyal, A., Sharma, P.K. and Kannan, M., 2025. *Exploratory Data Analysis (EDA) on Undergraduate Data Science Students Through R Programming*.
 - R Core Team (2024) *R: A Language and Environment for Statistical Computing*. Vienna: R Foundation for Statistical Computing. Available at: <https://www.R-project.org/>
 - Kuhn, M. 2008. Building Predictive Models in R Using the caret Package. *Journal of Statistical Software*, 28(5).
 - Kuhn, M., Weston, S., Culp, M., Coulter, N. and Quinlan, R., 2015. Package ‘C50’. CRAN, UTC.
 - Wickham, H. (2016) *ggplot2: Elegant Graphics for Data Analysis*. 2nd edn. New York: Springer.
 - Lantz, B. (2019) *Machine Learning with R*. 3rd edn. Birmingham: Packt Publishing.
-